ViMo: Generating Motions from Casual Videos

Liangdong Qiu[1], Chengxing Yu[2], Yanran Li, Zhao Wang[3], Haibin Huang[4], Chongyang Ma[5], Di Zhang[4], Pengfei Wan[4], and Xiaoguang Han[†1]

[1] Chinese University of Hong Kong (Shenzhen)
[2] University of Electronic Science and Technology of China
[3] Zhejiang University
[4] Kuaishou Technology
[5] ByteDance

Abstract. Although humans have the innate ability to imagine multiple possible actions from videos, it remains an extraordinary challenge for computers due to the intricate camera movements and montages. Most existing motion generation methods predominantly rely on manually collected motion datasets, usually tediously sourced from motion capture (Mocap) systems or Multi-View cameras, unavoidably resulting in a limited size that severely undermines their generalizability. Inspired by recent advance of diffusion models, we probe a simple and effective way to capture motions from videos and propose a novel Video-to-Motion-Generation framework (ViMo) which could leverage the immense trove of untapped video content to produce abundant and diverse 3D human motions. Distinct from prior work, our videos could be more causal, including complicated camera movements and occlusions. Striking experimental results demonstrate the proposed model could generate natural motions even for videos where rapid movements, varying perspectives, or frequent occlusions might exist. We also show this work could enable three important downstream applications, such as generating dancing motions according to arbitrary music and source video style. Extensive experimental results prove that our model offers an effective and scalable way to generate diversity and realistic motions. Code and demos will be public soon.

Keywords: Video-Motion Generation · Capture from Videos · Motion Diffusion Models

1 Introduction

Realistic 3D human motions play an important role as the 'soul' of virtual characters since they can make them feel alive – it enables the static character to jump, fight and dance [1]. With the rapid proliferation of portable devices and the metaverse, the demands on virtual character creation are rising at an explosive rate. In the traditional animation and film industry, the motions are created

[†] Corresponding author.

Fig. 1. An example of **Casual Video.** This is a game advertisement video. The camera's perspective and distance from the actor constantly change, with some joints being obscured. The character at the bottom is the 3D motion clips generated by our method, ViMo, which can extract comparable 3D motions from casual videos. Traditional human pose reconstruction methods often fails to obtain plausible motions on these casual videos where the cameras are complex and occlusion of characters exists from beginning to end.

manually in a tedious way either by expensive motion capture systems [35] or by professional animators, therefore usually resulting in a very time-consuming and laborious process. Motivated by this problem, data-driven motion generation methods [54,44,53,2,21,67] have been widely studied recently since they are super efficient and economical [64].

Although promising progress in motion generations has been achieved, the diversity and flexibility of the synthesised motions are still far behind the expectations of the users in practical applications. Most existing motion generation approaches adopt the diffusion-based, VAE or GAN-based framework and take condition signals which are input parameters or data types that guide the motion generation process, such as Gaussian noise [60,44], text [52,53,67], audio [29,2], music [3,50,23] or scene contexts [57,21]. The effectiveness of these models is contingent upon the availability of matched accurate examples (ground-truth pairs) of both the input conditions (like audio or text) and the corresponding motions. Consequently, this way narrows the synthesised motions only to specific domains and finite categories provided in the training dataset. For example, the state-of-the-artwork EDGE [54] generated dancing movements are all limited to the 10 categories of street dancing provided in the training dataset AIST++ [25]. The existing action-conditioned motion generations [11,31,42,15] are only able to generate motions in the same label of the input in the training dataset as well.

The motion datasets hamper these generation methods from scaling up. This is because they are typically captured using professional Mocap Systems or Multi-View cameras which are time-consuming and costly. In contrast, video resources are incredibly extensive and widely available, offering a broader range of diverse and interesting human actions. However, this topic remains very under-

exploited due to the significant complexity and variability of video content. There are some existing approaches [71,63,61,39,40,5] explored to leverage the 3D pose estimation methods to reconstruct the 3D human motions precisely from video resources, therefore generating more diverse types of motions. For example, The state-of-the-artwork MotionBERT [71] can estimate high-fidelity 3D joint positions and perform natural human movements similar to the videos. However, their models are mostly only tested on videos with specific conditions –the camera position could be known accurately. When the videos include complex camera movements or montage editing techniques (Fig 1), their performance could degrade drastically and fall into collapse. However, most online videos that we denote as **Casual videos** usually unavoidably contain complex camera movements, which are out of their assumption.

Reconstructing 3D motions from these casual videos will be incredibly complicated, as estimating the complex camera angles and locations is overly intricate and sometimes not feasible. Without accurate camera positions, the regression loss employed in the aforementioned methods will become completely dysfunctional and frame-wise poses are hard to string together as a smooth motion. Therefore, learning 3D motions from casual videos is a desirable goal but remains a formidable challenge due to the intricate camera issues.

In contrast, humans can easily comprehend 3D actions from casual videos. To capture the essence of actions in videos, the 3D motions just need to be similar in poses and rhythms rather than the high precision of joint locations. Inspired by that, we pose a new video-to-motion generation question – *could the computer generate multiple possible 3D motions of the new types according to the actions provided in the casual videos which have very similar pose shapes and rhythms?*

To achieve this goal, we propose a novel video-to-motion generation model by taking videos as conditions to generate various motions and showcase its great potential in enabling three important downstream applications. Specifically, we explore a diffusion-based framework which takes the 2D poses from multiple different views as the input data to generate 3D motions to see how far it could work on video-to-motion generation tasks. With this design, our model can create a sequence of 3D motion without explicitly estimating the camera positions. We conducted extensive experiments to validate the effectiveness of our approach. The results reveal that this simple approach could effectively generate unlimited complex and realistic 3D motions which are contextually consistent with the input casual videos even when they contain complex camera movements and occlusions. Notably, we show that our work could enable three interesting downstream applications. (1) Firstly, it could be an efficient way to produce large-scale motion datasets. We collected a large-scale Chinese classic dancing dataset and built up a 3D dancing motion dataset that included 750 motions. This new dataset could be used to facilitate data-driven tasks such as music-to-dancing generation, motion recognition and prediction. (2) Secondly, our model could power up few-shot stylization in the dancing generation. Specifically, we could leverage the music-to-motion generation pipeline to generate a specific style of dance according to the pose style offered by a few video examples. (3)

Thirdly, we could form a new video-guided motion completion and editing task which could fill in the missing part of the motion by the similar content defined by any source videos.

In summary, we contribute in the following aspects to encourage future research towards this goal: (1) We propose a novel video-to-motion-generation model to leverage the abundant resources of the actions in videos to enhance the diversity and realism of 3D motion generation. By posing this new question, we reveal that the diversity and flexibility of motion generation could be greatly extended by a simple but effective approach. (2) A dancing specified 3D motion dataset is constructed with the proposed ViMo model, which contains 52 Chinese classic dancing videos together with 750 generated 3D dancing motions. This demontrate the effectiveness and advantages of video-to-motion-generation strategy. We hope suck kind of video-to-motion dataset could facilitate a lot of motion-related tasks. (3) We demonstrate that the proposed method could be impressively activated in three real-life applications which illustrate its astonishing ability to generate realistic and aesthetical human motions.

2 Related Work

2.1 Human Motion Generation

Existing human motion generation proposes generative models which take conditional signals as guidance in order to consistently generate motions. These methods could be categorized based on different types of conditional signals, including Gaussian noise, seed motion, text, music, audio and scene contexts.

Action-conditioned motion generation sounds close to our work but they only take seed motions or an action label [11,31,42,15] to generate the motions in the same category of the reference action. In contrast, we could generate motions in an unseen category. Several work [60,44] have learned motion manifolds in which the user could sample the Gaussian noise as input to obtain new motions. However, their results on the manifold are mostly locomotion. Text-based motion generation [52,53,67] has attracted increasing attention recently due to the advanced natural language processing technologies. Such methods aim to learn an alignment embedding of the motion sequences and text description. Another stream of work generates hand gestures or dancing movements based on the audio data such as speech [69,29,62] and music [3,50,23]. Scene-guided motion generation [57,56,21] is highly goal-oriented and requires the generated motion to be physically reasonable and interactive with the objects in the scene environment such as indoor activities. Most of the aforementioned approaches are only able to synthesise motion categories which appeared in the training dataset before. Very few works such as MotionCLIP [52] and AvataCLIP [20] could leverage the CLIP loss to generate reasonable motion sequences from unseen categories with the text labels. However, there are unlimited motions like Kongfu, dancing, and sports that cannot be aligned with any category or precise text labels.

2.2 Motion Reconstruction from Video

The existing 2D to 3D pose estimation can be broadly categorized into two ways. The first type of methods [36,51,70] employ a convolutional neural network learning the image features from single image and regressing the 3D joints coordinates directly. The second type of methods [10,34,8,4,55,9,26,48,65,68,71,63,61,39,40,5] extract the 2D pose from the images then proposes diverse deep learning models to lift the 2D pose into 3D space, which is current most common way. State-of-the-art work MotionBERT [71] has provided a typical approach for 3D motion reconstruction by utilising the 3D pose estimation methods. They have trained a Dual-stream Spatio-temporal Transformer framework on a 3D motion regression task. Despite these deterministic methods, a diffusion-based 3D pose estimation framework [19] is proposed to generate stochastic multiple pose estimation results. Recently, SMPLer-X [5] has achieved the best performance on 3D pose estimation by scaling up the training data to be 4.5 million level.

Most of the aforementioned approaches achieve promising results by assuming the camera parameters are almost fixed or could be easily estimated accurately. For example, MotionBERT [71] and GLAMR [63] present very accurate motion reconstruction results when the camera is estimated correctly but the regression becomes completely infeasible when the camera is too complex. Furthermore, it remains a great challenge to compose the single skeletons together as a smooth and complete 3D motion even if they could show very accurate 3D estimation results on a single image, especially when the changes in camera position and orientation are significant.

3 Methodology

3.1 Overview

Given a video in the wild, which records a single person performing a certain action under various views and editing, video-to-motion generation aims to generate multiple plausible 3D human motions. The result motions are proposed to be semantically aligned to the video content in terms of pose shape and rhythm.

To address this issue, a diffusion-based framework ViMo is elaborated to conduct the video-to-motion generation. In the following paragraph, we further present the details of how to incorporate ViMo in three crucial downstream applications. Once the bridge of video content to 3D motion has been built, efficient and semantic control methods could be offered to the user therefore facilitating a more interactive experience in the motion applications.

3.2 Problem Definition

To simplify the scenario, the input casual video only contains a single person is considered. Taking existing pose extractors such as [13,7], the basic 2D pose estimation results could be obtained through the image features. The 2D pose sequences are denoted as $p \in \mathbb{R}^{S \times J_{2d} \times C_{\text{in}}}$. In this work, the input sequence length

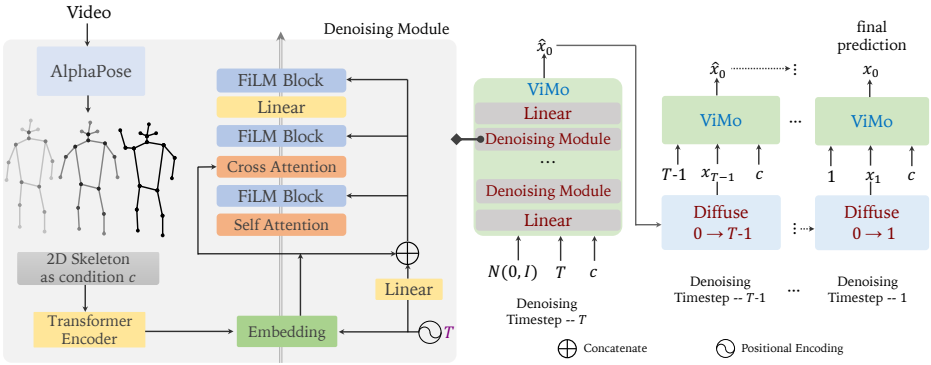

Fig. 2. Diagram of our ViMo **pipeline**. ViMo takes a sequence of 2D poses as input conditional signal c. Then it will process a denoise process to obtain a 3D motion sequence from time $t = T$ to $t = 0$. Note that the motion itself is the prediction at each denoising step. One of the advantages is the diffusion process performs robust on the casual videos and could generate corresponding 3D motions without estimating the precise camera positions.

is set as $S = 150$ and each pose in the sequences has a skeleton with 17 joints which could be written as $J_{2d} = 17, C_{\text{in}} = 3$.

Note the input 2D skeleton follows the standard COCO format [28] along with the predicted coordinates and confidence. The missing joints and low-confidence joints will be filtered out in the proposed pipeline. Our objective is to generate 3D motion $\mathbf{M}$ as a sequence of joints rotations $R \in \mathbb{R}^{J_{3d} \times C}$ in 6D rotation space, foot contact labels $\mathbb{R}^4$ and root position $\mathbb{R}^3$ where $J_{3d} = 24, C = 6$.

Short technical summaries (what these extractors output and how)

OpenPose [7] − bottom-up, Part Affinity Fields

- **What it outputs**: per-frame 2D keypoint coordinates for body (and optionally face/hands/feet), confidence/score maps for each keypoint, and Part-Affinity Fields (PAFs) that encode which keypoints belong to the same person. OpenPose produces standard keypoint locations + confidence values.
- **How it works** (brief): bottom-up network predicts heatmaps (per-part confidence) and PAF vector fields; a parsing step groups parts into person skeletons. It's real-time and widely used as a general 2D keypoint extractor.

AlphaPose [13] − high-accuracy whole-body, tracking, top-down style system

What it outputs: 2D keypoint coordinates and confidence for whole-body keypoints (body + hands + face + feet), and it includes tracking (PoseFlow / identity embeddings) to keep person IDs across frames. AlphaPose can supply COCO-style body keypoints (17 joints) or denser whole-body keypoints depending on configuration.

How it works: AlphaPose uses techniques such as Symmetric Integral Keypoint Regression (SIKR), parametric pose NMS (P-NMS) and identity embeddings for tracking; it is designed to be fast and accurate for multi-person, whole-body estimation.

Exact COCO 17 keypoint order (to match ViMo's J2d = 17)

COCO's standard keypoint indices map to body parts in this order: [nose, left_eye, right_eye, left_ear, right_ear, left_shoulder, right_shoulder, left_elbow, right_elbow, left_wrist, right_wrist, left_hip, right_hip, left_knee, right_knee, left_ankle, right_ankle]

What the visibility 'v' means (COCO annotation semantics)

COCO encodes '*v*' as an integer indicating whether a keypoint was labelled and whether it was visible in the annotation:

- v = 0 ⟶ keypoint not labeled / out of image (not annotated).
- v = 1 ⟶ keypoint labeled but not visible (occluded / not clearly visible).
- v = 2 ⟶ keypoint labeled and visible.

This is COCO's annotation visibility flag, not a detector confidence score. COCO's *v* is a ground-truth annotation visibility/label flag stored with dataset annotations.

- Pose detectors (OpenPose / AlphaPose / others) produce predicted (x,y) coordinates and a confidence/score for each predicted keypoint (a float probability / score). ViMo's phrasing "predicted coordinates and confidence" refers to those detector outputs (per-keypoint scores), not to the COCO annotation *v*.
- So, in practice: when ViMo says COCO format + predicted coordinates and confidence, the pipeline is usually: Run AlphaPose/OpenPose ⟶ get per-frame (x,y,score) per keypoint (detector output). Pack those into the same per-keypoint structure (x, y, confidence) aligned to the COCO joint ordering to feed the model.

The shape m ∈ $R^{\{S \times (24\times6+4+3)\}}$ — decode the numbers (step-by-step)

ViMo writes the target 3D motion vector per frame as having three parts:

1. Joint rotations in 6D:

- J3d = 24 (they use 24 joints in the 3D skeleton).
- C = 6 ⟶ each joint rotation is represented by 6 numbers (6D rotation representation packs each joint's 3D orientation into 6 numbers in a way that's continuous and easy for neural nets to learn, refer "On the Continuity of Rotation Rep.s in Neural Network" paper).
- So joint-rotation portion per frame = 24 × 6 = 144 numbers.

2. Foot contact labels (R^4):

A short binary vector of length 4 (so +4). ViMo says they precompute binary foot contact labels $\{0,1\}^{\{N\times4\}}$ from foot/toe velocity near zero (i.e., whether each foot or toe is in contact with ground). This is important for physical plausibility (foot sliding, planting).

3. Root position (R^3):

A 3D global root position (x,y,z) for the body root (global translation), so +3. ViMo explicitly stores the global position of the root joint along with rotation.

*Add them up: 144 (rotations) + 4 (foot) + 3 (root) = 151 scalars per frame. If sequence length is S frames, the full motion is S × 151. ViMo denotes this compactly as S × (24*6+4+3).*

Foot contact labels and root position — made easy

- **Foot contact labels (4 numbers):** these are binary flags telling whether each foot/tip is touching the ground at that frame (e.g., left foot, right foot, left toe, right toe) − used to avoid feet sliding and to make motion physically plausible. ViMo computes them from the foot/toe vertical velocity being near zero.
- **Root position (3 numbers):** the global (x,y,z) location of the body's root (usually pelvis or hips). This controls where the character is in world space so the motion is not just orientation but also translation.

3.3 ViMo framework

Diffusion Models. Inspired by recent successes in image generation and natural language processing tasks [47,46,6,59,66,27], a transformer based diffusion model is leveraged to align the synthesis motion to the 2D pose sequences. Considering the generation as a Markov noising Process following [18], the 2D pose sequence p would join the noising process to finally produce a 3D motion $m \in \mathbb{R}^{S\times(24\times6+4+3)}$ corresponding to the conditional signal.

A typical diffusion process is employed as an initial step. The noising process satisfies

$$q\left(m_t^{1:S} \mid m_{t-1}^{1:S}\right) = \mathcal{N}\left(\sqrt{\alpha_t} m_{t-1}^{1:S}, (1-\alpha_t)\, I\right), \tag{1}$$

with latent $\left\{m_t^{1:S}\right\}_{t=0}^{T}$ which is assumed as Gaussian distribution and $x_0^{1:S}$ is sampled from original data $m \sim q\left(\mathbf{M} \mid p\right)$. $\alpha_t \in (0,1)$ is the constant parameter at noising step $t \in [0,1,...,T]$.

The reversed denoising process is modelled with a neural network starting from the Gaussian distribution $m_T^{1:S} \sim \mathcal{N}(0,I)$ with simple objective [18]

$$\hat{m}_0 = D\left(m_t, t, p\right), \tag{2}$$

$$\mathcal{L}_{\text{simple}} = E_{m\sim q(\mathbf{M}|p),t\sim[1,N]}\left[\left\|m_0 - D\left(m_t, t, c\right)\right\|_2^2\right], \tag{3}$$

where $\hat{m}_0$ obeys the original data distribution $q\left(\mathbf{M} \mid p\right)$. It is used for computing the posterior distribution $q\left(m_{t-1} \mid m_t, m_0, p\right)$ at denoising step t [45]. It needs to point out that result motions are directly predicted rather than reparameterization of vanilla epsilon.

Network Architecture. The schematic of the proposed network architecture and denoising steps are described in Fig 2. Given an input m_t, three denoising modules would gradually recover the data by reducing the noise from time step t to $t-1$. Each module consists of basic attention blocks and MLP layers. The first self-attention block is designed to leverage the information of the input m_t. During the diffusion process, the time step and 2d poses p feature are incorporated together by FiLM blocks [41]. After that, the fused feature vector is sent into cross-attention and MLP layers to enhance the capability of the model. Note that, sampling processing is employed during training in a classifier-free manner and Classifier-free diffusion guidance. The resulting estimate is composed of 25% unconditioned guidance and 75% conditioned guidance, balancing between diversity and fidelity by allowing a 25% probability to drop off the guidance.

Overview:

- **Diffusion models (Denoising Diffusion Probabilistic Model - DDPMs)**: imagine you take a clean motion (a whole mocap clip) and progressively add a little Gaussian noise to it over many steps until it becomes almost random noise. That's the **forward / noising** Markov process.
- Then you train a neural network to **reverse** that corruption: starting from pure noise, step-by-step remove noise until you get back a plausible clean motion. That's the **reverse / denoising** learned process.
- Conditioning: ViMo gives the network extra input — the **2D pose sequence** from the video — so the denoiser produces 3D motion that matches the 2D pose rhythm/shape.
- Important: the diffusion **time-step** 't' here means *how noisy* a sample is (not the video time axis). The whole motion sequence (many frames) is corrupted together at each diffusion level and denoised together — the model is sequence-aware (transformer) and predicts the full 3D motion sequence from noisy input plus 2D condition.

What is a Markov noising process?

- **Markov**: each noisy state only depends on the previous noisy state (not the entire history). Formally: $q(m_t \mid m_{t-1}, m_{t-2}, \dots) = q(m_t \mid m_{t-1})$.
- **Noising**: at each step we add Gaussian noise with a prescribed variance schedule. Over many steps the clean data becomes dominated by noise.

The forward (noising) process:

Notation adapted to ViMo's m (motion) variable:

- Let m_0 be a clean 3D motion sample (a whole sequence represented as a vector; ViMo uses $m_0 \in \mathbb{R}^{S\times 151}$, i.e., S frames × 151 scalars/frame; more on that later).
- We define a sequence $m_1, m_2, \dots, m_T$ where

$$q(m_t \mid m_{t-1}) = \mathcal{N}\big(\sqrt{\alpha_t}\, m_{t-1},\ (1-\alpha_t)\, I\big), \qquad t = 1, \dots, T.$$

This is exactly the equation ViMo wrote (their Eqn (1)).

Interpretation: each step shrinks the signal by factor $\sqrt{\alpha_t}$ and adds isotropic Gaussian noise with variance $1-\alpha_t$. The parameters $\{\alpha_t\}$ (usually $0 < \alpha_t < 1$) are chosen so that after many steps m_T is nearly standard Gaussian.

Closed-form expression for m_t in terms of m_0

Because the forward transitions are Gaussian and linear, we can write m_t directly as:

$$m_t = \sqrt{\bar\alpha_t}\, m_0 \ + \ \sqrt{1-\bar\alpha_t}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I),$$

where

$$\bar\alpha_t \ = \ \prod_{s=1}^{t} \alpha_s.$$

Derivation sketch (inductive):

- For $t = 1$: $m_1 = \sqrt{\alpha_1} m_0 + \sqrt{1-\alpha_1}\epsilon_1$.
- Assume $m_{t-1} = \sqrt{\bar\alpha_{t-1}} m_0 + \sqrt{1-\bar\alpha_{t-1}}\epsilon'$.
- Substitute into $m_t = \sqrt{\alpha_t} m_{t-1} + \sqrt{1-\alpha_t}\epsilon_t$. After algebra, that becomes the stated closed form with $\bar\alpha_t = \alpha_t \bar\alpha_{t-1}$ and a single Gaussian ϵ (since linear combination of independent Gaussians is Gaussian).

how?

Why $\sqrt{\alpha}$ appears: scaling the *signal* by $\sqrt{\alpha_t}$ controls how much the original m_0 survives at step t. The variance of the noise term must be $1-\bar\alpha_t$ to match the total variance; square roots show up because the forward transition acts on the mean linearly (so the multiplicative factors are square roots).

Derivation of the closed-form forward noising formula:

Induction step: assume true for $t-1$, prove for t

Induction hypothesis. Suppose for some $t - 1 \geq 1$,

$$m_{t-1} \ = \ \sqrt{\bar\alpha_{t-1}}\, m_0 \ + \ \sqrt{1-\bar\alpha_{t-1}}\, \epsilon', \qquad \epsilon' \sim \mathcal{N}(0, I),$$

where $\bar\alpha_{t-1} = \prod_{s=1}^{t-1} \alpha_s$. (Here ϵ' is a Gaussian formed from the independent noises used up to step $t-1$.)

One step update. By the forward rule,

$$m_t \ = \ \sqrt{\alpha_t}\, m_{t-1} \ + \ \sqrt{1-\alpha_t}\, \epsilon_t, \qquad \epsilon_t \sim \mathcal{N}(0, I),$$

with ϵ_t independent from ϵ' and from m_0.

Substitute the induction hypothesis for m_{t-1}:

$$\begin{aligned} m_t &= \sqrt{\alpha_t}\Big(\sqrt{\bar\alpha_{t-1}}\, m_0 + \sqrt{1-\bar\alpha_{t-1}}\, \epsilon'\Big) + \sqrt{1-\alpha_t}\, \epsilon_t \\ &= \sqrt{\alpha_t\bar\alpha_{t-1}}\, m_0 \ + \ \sqrt{\alpha_t}\sqrt{1-\bar\alpha_{t-1}}\, \epsilon' \ + \ \sqrt{1-\alpha_t}\, \epsilon_t. \end{aligned}$$

Recognize $\alpha_t\bar\alpha_{t-1} = \bar\alpha_t$. So the deterministic (mean) part is:

$$\sqrt{\alpha_t\bar\alpha_{t-1}}\, m_0 = \sqrt{\bar\alpha_t}\, m_0.$$

The remaining terms are a linear combination of two independent zero-mean Gaussians:

$$\eta \ := \ \sqrt{\alpha_t}\sqrt{1-\bar\alpha_{t-1}}\, \epsilon' \ + \ \sqrt{1-\alpha_t}\, \epsilon_t.$$

So

$$m_t \ = \ \sqrt{\bar\alpha_t}\, m_0 \ + \ \eta.$$

We need to show η has the form $\sqrt{1-\bar\alpha_t}\, \epsilon$ for some $\epsilon \sim \mathcal{N}(0, I)$.

Because ϵ' and ϵ_t are independent and each has covariance I,

$$\mathrm{Cov}(\eta) = \big(\alpha_t(1-\bar\alpha_{t-1})\big) I \ + \ \big(1-\alpha_t\big) I = \big(\alpha_t - \alpha_t\bar\alpha_{t-1} + 1 - \alpha_t\big) I.$$

Simplify the scalar:

$$\alpha_t - \alpha_t\bar\alpha_{t-1} + 1 - \alpha_t = 1 - \alpha_t\bar\alpha_{t-1} = 1 - \bar\alpha_t.$$

So

$$\mathrm{Cov}(\eta) = (1-\bar\alpha_t)\, I.$$

Thus η is zero-mean Gaussian with covariance $(1-\bar\alpha_t) I$. Equivalently,

$$\eta \overset{d}{=} \sqrt{1-\bar\alpha_t}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).$$

(Here $\overset{d}{=}$ means equality in distribution. Concretely we can define $\epsilon := \eta / \sqrt{1-\bar\alpha_t}$, and ϵ will be $\mathcal{N}(0, I)$.)

Putting the mean and noise together:

$$m_t \ = \ \sqrt{\bar\alpha_t}\, m_0 \ + \ \sqrt{1-\bar\alpha_t}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).$$

Key observations:

- **Why sqrt(α_t) multiplies the previous mean:** the forward Gaussian transition scales the mean (signal) by sqrt(α_t) at each step; sequential application multiplies these scalings, producing sqrt($\bar\alpha_t$) in the closed form.
- **Why the noise combines into a single Gaussian:** linear combinations of independent Gaussian variables are Gaussian; their covariance adds. The specific scalar sqrt(1–$\bar\alpha_t$) is forced by matching the total variance to 1–$\bar\alpha_t$.
- **Practical meaning:** at diffusion step 't', a fraction $\bar\alpha_t$ of the original signal survives in the mean, and the remaining variance 1–$\bar\alpha_t$ is pure Gaussian noise.

clarify notation:

About "sampled $x^0_{1:S}$" in the ViMo text — their typesetting is a bit inconsistent: they use $x_0^{1:S}$ or $m_0^{1:S}$ to denote the full sequence of frames for S frames. In short: m_0 is the whole-sequence ground truth motion; if you prefer to see the per-frame view: $m_0 = [m_{0,1}, m_{0,2}, \dots, m_{0,S}]$. ViMo uses the whole sequence as a single data vector in diffusion.

Where does the 3D ground truth M come from (if dataset is videos)?

- **ViMo's training requires *paired* data**: videos **with corresponding 3D motions**. In their experiments they train on datasets that *do* provide 3D mocap aligned with videos
- For videos that **don't** have ground-truth 3D (the casual videos the paper targets), ViMo's goal is to **generate** plausible 3D motions conditioned on the video's 2D poses
- In short: training uses datasets with GT 3D; inference can run on plain videos (2D poses only) to produce 3D.
- So, when the paper writes **m ~ q(M|p)** they mean: the data distribution over 3D motions conditional on 2D pose sequences (during training we have examples from real dataset pairs (p, m0).

The reverse (denoising) process and the posterior $q(m_{t-1}|m_t,m_0)$:

Because the forward process is Gaussian and linear, the true posterior $q(m_{t-1}|m_t,m_0)$ is also Gaussian** with known mean and variance. That posterior is used in deriving what the neural denoiser should predict. (** how? Refer below)

Using the standard DDPM formulas: (explained below)

- Posterior mean:

$$\tilde{\mu}_t(m_t, m_0) \;=\; \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t}\, m_0 \;+\; \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\, m_t,$$

- Posterior variance:

$$\tilde{\beta}_t \;=\; \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\,\beta_t,$$

where $\beta_t = 1-\alpha_t$.

This is the exact Gaussian $q(m_{t-1} \mid m_t, m_0) = \mathcal{N}(\tilde{\mu}_t, \tilde{\beta}_t I)$. The denoiser network **approximates** the reverse conditional $p_\theta(m_{t-1} \mid m_t, p)$, usually by predicting parameters (a mean) used to sample the next less-noisy state.

Takeaway: the posterior gives a *ground-truth* target for the reverse step if you know m_0. In training, that forms the basis for objectives.

Derivation of the posterior:

- The *data* is the entire motion sequence vector m_0. At diffusion step t we have noisy m_t.
- Forward/"noising" rule (ViMo / DDPM style):

$$q(m_t \mid m_{t-1}) \;=\; \mathcal{N}\big(\sqrt{\alpha_t}\, m_{t-1},\; \beta_t I\big) \quad \text{where} \quad \beta_t \equiv 1-\alpha_t.$$

- Also from repeated application we have the shorthand

$$\bar{\alpha}_t \;=\; \prod_{s=1}^{t} \alpha_s.$$

- The prior on m_{t-1} given m_0 (i.e., result of forward process up to $t-1$) is:

$$q(m_{t-1} \mid m_0) \;=\; \mathcal{N}\big(\sqrt{\bar{\alpha}_{t-1}}\, m_0,\; (1-\bar{\alpha}_{t-1})I\big).$$

(This comes from the closed-form forward expression $m_{t-1} = \sqrt{\bar{\alpha}_{t-1}} m_0 + \sqrt{1-\bar{\alpha}_{t-1}}\,\epsilon$.)

We want $q(m_{t-1} \mid m_t, m_0)$. Use Bayes:

$$q(m_{t-1} \mid m_t, m_0) = \frac{q(m_t \mid m_{t-1})\, q(m_{t-1} \mid m_0)}{q(m_t \mid m_0)}.$$

When treating this as a function of the unknown m_{t-1}, the denominator $q(m_t|m_0)$ does **not depend** on m_{t-1} (it depends only on m_t, m_0), so it is a **constant factor** — it only changes the normalizing constant of the posterior, not its shape (mean/variance). Therefore, we can write:

$$q(m_{t-1} \mid m_t, m_0) \;\propto\; q(m_t \mid m_{t-1})\, q(m_{t-1} \mid m_0),$$

and determine the posterior mean and covariance by matching the quadratic and linear terms in the exponent (completing the square). The normalization constant only matters if you need the exact probability value, not the mean/covariance.

Remember: Normal Distribution Formula:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

Normalization const --> $e^{-1/2}$/sigma*sqrt(2*pi) is not considered here.

Scalar case — explicit arithmetic:

Let $x := m_{t-1}$ (unknown scalar). We have:

- **Prior** $q(x \mid m_0)$ is Gaussian with mean μ_p and variance σ_p^2:

$$q(x \mid m_0) \;=\; \mathcal{N}(x; \mu_p, \sigma_p^2) \quad \text{so} \quad q(x \mid m_0) \propto \exp\Big(-\frac{(x-\mu_p)^2}{2\sigma_p^2}\Big).$$

  In DDPM notation $\mu_p = \sqrt{\bar{\alpha}_{t-1}}\, m_0$ and $\sigma_p^2 = 1-\bar{\alpha}_{t-1}$.
- **Likelihood** $q(m_t \mid x)$ is Gaussian in m_t with mean $\sqrt{\alpha_t}\, x$ and variance β_t:

$$q(m_t \mid x) \propto \exp\Big(-\frac{(m_t - \sqrt{\alpha_t}x)^2}{2\beta_t}\Big).$$

Multiplying two Gaussians (in the same variable) gives another Gaussian. To see its mean/variance, we just expand the exponent.

Posterior (up to constant):

$$q(x \mid m_t, m_0) \;\propto\; \exp\Big(-\frac{(x-\mu_p)^2}{2\sigma_p^2}\Big)\, \exp\Big(-\frac{(m_t - \sqrt{\alpha_t}x)^2}{2\beta_t}\Big).$$

Take the exponent (call it $-E(x)$), and expand both squares:

$$\begin{aligned} E(x) &= \frac{(x-\mu_p)^2}{2\sigma_p^2} + \frac{(m_t - \sqrt{\alpha_t}x)^2}{2\beta_t} \\ &= \frac{1}{2}\Big[\underbrace{\Big(\frac{1}{\sigma_p^2} + \frac{\alpha_t}{\beta_t}\Big)}_{=:A} x^2 \;-\; 2\underbrace{\Big(\frac{\mu_p}{\sigma_p^2} + \frac{\sqrt{\alpha_t}\, m_t}{\beta_t}\Big)}_{=:B} x + (\text{terms independent of } x)\Big]. \end{aligned}$$

We know Gaussians look like $\mathcal{N}(x; \mu, \sigma^2) \;\propto\; \exp\Big(-\frac{(x-\mu)^2}{2\sigma^2}\Big)$. So, if we want to show that our posterior is Gaussian, we must massage the exponent E(x) until it has the form $\frac{(x-\mu_{\text{post}})^2}{2\sigma_{\text{post}}^2}$. That means: **get everything into "a square shifted around the mean."**

Right now, $E(x)$ is messy:

$$E(x) = \tfrac{1}{2}Ax^2 - Bx + \text{const}.$$

- The Ax^2 part suggests variance (spread).
- The $-Bx$ part suggests a shift of the mean.
  But it's not in a neat $(x-\mu)^2$ shape yet.

So we **complete the square**: algebra that bundles the x^2 and x terms into one square:

$$\tfrac{1}{2}Ax^2 - Bx = \tfrac{1}{2}A(x - B/A)^2 - \tfrac{1}{2}A(B/A)^2.$$

Now the coefficient inside the square tells us the mean:

$$\mu_{\text{post}} = \frac{B}{A}.$$

And the prefactor of the square tells us the variance:

$$\sigma^2_{\text{post}} = \frac{1}{A}.$$

Simply put (another way - easier):

We want to rewrite it as

$$E(x) = \tfrac{1}{2}A(x - \mu_{\text{post}})^2 + \text{const}.$$

Expand the RHS:

$$\tfrac{1}{2}A(x-\mu_{\text{post}})^2 = \tfrac{1}{2}A(x^2 - 2\mu_{\text{post}}x + \mu^2_{\text{post}}) = \tfrac{1}{2}Ax^2 - A\mu_{\text{post}}x + \tfrac{1}{2}A\mu^2_{\text{post}}.$$

Compare with $E(x) = \tfrac{1}{2}Ax^2 - Bx + \text{const}$.

👉 Match coefficients:

- Quadratic matches automatically.
- Linear: $-A\mu_{\text{post}} = -B \;\Rightarrow\; \mu_{\text{post}} = B/A$.

So the posterior mean is $\mu_{\text{post}} = B/A$.

And since Gaussian variance is the inverse of the quadratic coefficient:
posterior variance = $1/A$.

Why is this valid?

Because the exponential of a quadratic function *is always Gaussian*. Completing the square is just a way of **identifying the mean and variance** hidden inside that quadratic.

The extra constant (like $-\frac{1}{2}A(B/A)^2$) only changes the overall scaling of the density. That scaling gets absorbed into the normalizing constant so the PDF integrates to 1. It doesn't affect mean/variance.

From the scalar derivation we used:

$$A \;=\; \frac{1}{\sigma_p^2} + \frac{\alpha_t}{\beta_t}, \qquad B \;=\; \frac{\mu_p}{\sigma_p^2} + \frac{\sqrt{\alpha_t}\, m_t}{\beta_t},$$

where, in DDPM notation,

$$\mu_p = \sqrt{\bar{\alpha}_{t-1}}\, m_0, \qquad \sigma_p^2 = 1-\bar{\alpha}_{t-1}, \qquad \beta_t = 1-\alpha_t.$$

Substitute σ_p^2 and μ_p into A and B:

$$A = \frac{1}{1-\bar{\alpha}_{t-1}} + \frac{\alpha_t}{\beta_t}, \qquad B = \frac{\sqrt{\bar{\alpha}_{t-1}}\, m_0}{1-\bar{\alpha}_{t-1}} + \frac{\sqrt{\alpha_t}\, m_t}{\beta_t}.$$

COMPUTE 1/A (POSTERIOR VARIANCE):

Put the two terms in A over a common denominator:

$$A = \frac{1}{1-\bar{\alpha}_{t-1}} + \frac{\alpha_t}{\beta_t} = \frac{\beta_t}{\beta_t(1-\bar{\alpha}_{t-1})} + \frac{\alpha_t(1-\bar{\alpha}_{t-1})}{\beta_t(1-\bar{\alpha}_{t-1})}.$$

Combine numerators:

$$A = \frac{\beta_t + \alpha_t(1-\bar{\alpha}_{t-1})}{\beta_t(1-\bar{\alpha}_{t-1})}.$$

Simplify the numerator:

$$\beta_t + \alpha_t(1-\bar{\alpha}_{t-1}) = (1-\alpha_t) + \alpha_t - \alpha_t\bar{\alpha}_{t-1} = 1 - \alpha_t\bar{\alpha}_{t-1}.$$

But $\alpha_t\bar{\alpha}_{t-1} = \bar{\alpha}_t$. So numerator $= 1 - \bar{\alpha}_t$. Thus

$$A = \frac{1-\bar{\alpha}_t}{\beta_t(1-\bar{\alpha}_{t-1})}.$$

Therefore

$$\frac{1}{A} = \frac{\beta_t(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}.$$

This is exactly the DDPM posterior variance:

$$\boxed{\tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t.}$$

(Recall $\tilde{\beta}_t$ is the scalar variance so posterior covariance $= \tilde{\beta}_t I$.)

COMPUTE B/A (POSTERIOR MEAN):

We have

$$\frac{B}{A} = \left(\frac{1}{A}\right) B = \frac{\beta_t(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\left(\frac{\sqrt{\bar{\alpha}_{t-1}}\, m_0}{1-\bar{\alpha}_{t-1}} + \frac{\sqrt{\alpha_t}\, m_t}{\beta_t}\right)$$

Distribute the prefactor across the sum (two terms):

- First term:

$$\frac{\beta_t(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t} \cdot \frac{\sqrt{\bar{\alpha}_{t-1}}\, m_0}{1-\bar{\alpha}_{t-1}} = \frac{\sqrt{\bar{\alpha}_{t-1}}\, m_0\, \beta_t}{1-\bar{\alpha}_t}.$$

- Second term:

$$\frac{\beta_t(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t} \cdot \frac{\sqrt{\alpha_t}\, m_t}{\beta_t} = \frac{\sqrt{\alpha_t}\, m_t\, (1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}.$$

So adding both:

$$\frac{B}{A} = \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1-\bar{\alpha}_t} m_0 + \frac{\sqrt{\alpha_t}\,(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t} m_t.$$

Thus we obtain the DDPM posterior mean:

$$\boxed{\tilde{\mu}_t(m_t, m_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1-\bar{\alpha}_t} m_0 + \frac{\sqrt{\alpha_t}\,(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t} m_t.}$$

Two different kinds of "constants"

- **Constant added *outside* the exponential (to the random variable):**
  Example: $\mathcal{N}(x;\mu,\sigma^2)$ vs. $\mathcal{N}(x+c;\mu,\sigma^2)$.
  👉 This *does* shift the mean, because you are literally shifting the variable x.
- **Constant added inside the exponent but independent of x:*
  Example:

$$p(x) \propto \exp\Big(-\frac{(x-3)^2}{2}\Big) \quad \text{vs.} \quad p(x) \propto \exp\Big(-\frac{(x-3)^2}{2} - 10\Big).$$

  👉 Here, the "-10" multiplies the whole distribution by e^{-10}. That just rescales the height of the curve, not its shape. When you normalize the distribution so that $\int p(x)\,dx = 1$, that constant disappears. The mean/variance are unchanged.

Key point: At no place did we need the marginal $q(m_t|m_0)$ because we were only manipulating the exponent (which depends on x) — the marginal is a multiplicative constant independent of x.

Vector case (matrix notation) — same idea but compact:

Let x be vector m_{t-1}. Use covariance matrices:

- Prior: $q(x \mid m_0) = \mathcal{N}(x;\mu_p,\Sigma_p)$ where $\mu_p = \sqrt{\bar{\alpha}_{t-1}}\, m_0$ and $\Sigma_p = (1-\bar{\alpha}_{t-1})I$.
- Likelihood: $q(m_t \mid x) = \mathcal{N}(m_t; \sqrt{\alpha_t}x, \beta_t I)$.

Form exponent (dropping constants):

$$E(x) = \tfrac{1}{2}(x-\mu_p)^\top \Sigma_p^{-1}(x-\mu_p) + \tfrac{1}{2}(m_t - \sqrt{\alpha_t}x)^\top(\beta_t I)^{-1}(m_t - \sqrt{\alpha_t}x).$$

Expand and collect the quadratic term in x:

$$\text{Quadratic (precision) matrix } A = \Sigma_p^{-1} + \frac{\alpha_t}{\beta_t} I = \frac{1}{1-\bar{\alpha}_{t-1}} I + \frac{\alpha_t}{\beta_t} I.$$

Linear vector (the d in $\frac{1}{2}x^\top Ax - x^\top d$):

$$d = \Sigma_p^{-1}\mu_p + \frac{\sqrt{\alpha_t}}{\beta_t} m_t = \frac{\sqrt{\bar{\alpha}_{t-1}}}{1-\bar{\alpha}_{t-1}} m_0 + \frac{\sqrt{\alpha_t}}{\beta_t} m_t.$$

Complete the square: posterior covariance $\Sigma_{\text{post}} = A^{-1}$, posterior mean $\mu_{\text{post}} = A^{-1}d$. Carrying out the algebra (same simplifications as in scalar case) yields the earlier vector formulas:

$$\tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t, \qquad \tilde{\mu}_t = \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1-\bar{\alpha}_t} m_0 + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t} m_t.$$

Training — epsilon-prediction vs x_0-prediction

This is where many papers differ; ViMo's text explicitly says *they predict the motion itself directly*, not a re-parameterized epsilon. Let me explain both formulations and how they relate.

A — The original DDPM (predict ϵ)

- Plug the closed form $m_t = \sqrt{\bar{\alpha}_t}\, m_0 + \sqrt{1-\bar{\alpha}_t}\,\epsilon$.
- If you train a network to **predict the noise** ϵ from m_t, the training loss is:

$$L_{\text{simple}}(\theta) = \mathbb{E}_{m_0,\epsilon,t}\Big[\big\|\epsilon - \epsilon_\theta(m_t, t, p)\big\|^2\Big],$$

where ϵ_θ is the network's output. This is the objective used in Ho et al. It tends to be numerically stable and has good properties.

Given ϵ_θ, you can estimate m_0 as

$$\hat{m}_0 = \frac{1}{\sqrt{\bar{\alpha}_t}}\big(m_t - \sqrt{1-\bar{\alpha}_t}\,\epsilon_\theta(m_t, t, p)\big).$$

From $\hat{m}_0$ you can compute the posterior mean and sample m_{t-1}.

(just substitute ϵ_θ in closed form eq. along with all other known parameters and m_0 we obtain is the estimate of the ground truth)

B — Predict m_0 (the clean data) directly

- Alternative objective: train the network to **predict** m_0 **directly**:

$$L_{\text{simple}}(\theta) = \mathbb{E}_{m_0,t,\epsilon}\big[\|m_0 - m_{0,\theta}(m_t, t, p)\|^2\big].$$

This is what ViMo states they do: "result motions are directly predicted rather than reparameterization of vanilla epsilon." In other words, D(mt,t,p) outputs m^0 directly. The paper uses that L_{simple} form as exactly in Eqn (3).

Relation between the two: If you predict ϵ\epsilon, you can compute m^0; if you predict m^0, you can compute an implied ϵ^. They are convertible using the closed-form relation shown above. The choice (predict ϵ vs predict m_0) is mostly an implementation/training-stability choice — some tasks benefit from direct m0m_0m0 prediction because the target data structure is complex and you want the network's last layer to output the clean structured data directly.

Why ViMo might prefer predicting m_0: The target is a structured, multi-part vector per frame (6D rotations, foot contacts, root pos). Predicting the whole structured m_0 directly may make it easier to include auxiliary losses (FK loss, velocity loss, foot loss — ViMo uses those) that are computed on the predicted motion itself. If the network predicts m^0 directly, those auxiliary losses can be applied straightforwardly. ViMo does combine the diffusion L_{simple} with auxiliary FK/vel/foot losses in Eqn (7).

"Reparameterization of vanilla epsilon" — what that means?

- "Vanilla epsilon" means the original DDPM parameterization where the network predicts the noise ϵ.
- A "reparameterization" refers to expressing the network's output in a different variable (e.g., predicting m_0 instead of ϵ) — they are interchangeable because of the closed-form relation between m_t, m_0 and ϵ.
- ViMo is telling you: *we do not use the usual ϵ-prediction formulation; we directly predict m_0.* That's just a modeling choice.

Does D(m_t,t,p) output a "less noisy 3D mocap" and is that the posterior?

- D(m_t,t,p) outputs **an estimate of the clean motion m^$_0$** (in ViMo's design). From m^$_0$ and the known forms (the posterior formula), we can compute the parameters for $p_\theta(m_{t-1}|m_t,p)$ (the denoising conditional distribution) and sample m_{t-1}.
- So: **No**, the network does not directly output the true posterior $q(m_{t-1}|m_t,m_0)$. It outputs a quantity (here m^$_0$) that we can plug into the analytic Gaussian posterior formula to get a mean for m_{t-1}. The actual posterior is known analytically only because the forward process is Gaussian and we know m_0 exactly in training; the denoiser approximates it at sampling time via its predicted m^$_0$.

Process per denoising step (high-level):

1. From current noisy sample m_t, network outputs $\hat{m}_0 = D(m_t, t, p)$.
2. Compute posterior mean $\tilde{\mu}_t(m_t, \hat{m}_0)$ via the formula above.
3. Sample $m_{t-1} \sim \mathcal{N}(\tilde{\mu}_t, \tilde{\beta}_t I)$, or take the mean (deterministic DDIM variants)
4. Repeat until $t = 0$ to obtain final $\hat{m}_0$.

— this yields the next less-noisy sample.

Derivation example - to sample m_{t-1}

Given the known closed form for the forward process and the analytic posterior, the sampling step typically uses the network-predicted $\hat{m}_0$:

1. Suppose we have m_t and we compute $\hat{m}_0 = D(m_t, t, p)$.
2. Compute

$$\tilde{\mu}_t = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t}\hat{m}_0 + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t} m_t.$$

3. Sample $m_{t-1} \sim \mathcal{N}(\tilde{\mu}_t, \tilde{\beta}_t I)$. (Or use deterministic DDIM formulas to avoid sampling noise at each step.)
   This is what the reverse chain does at generation time.

What is "time step" t in diffusion, and how does it relate to video frames?

- The diffusion **time-step** t ∈ {1, …, T} is **not** the video frame index. It is the index in the *noise schedule* that controls how strongly the data has been corrupted.
- Video frames (frame index) live inside the single data vector m_0 (which encodes a whole sequence of S frames). The diffusion process corrupts the **entire motion sequence** m_0 (all frames together) to produce m_t. So, at each diffusion index t, you have a noisy *sequence* m_t representing the whole clip.
- **Why corrupt the whole sequence at once?** This allows the denoiser to exploit *temporal* dependencies (smoothness, rhythm) while denoising. If you noised and denoised frames independently you'd lose important temporal coherence.

FiLM Blocks (Feature-wise Linear Modulation)

A FiLM (Feature-wise Linear Modulation) layer is a **dynamic** form of a linear layer. Instead of having fixed weights, its "weights" are *generated on-the-fly* by a separate network based on an external condition.

A FiLM layer applies an *affine transformation* (scale and shift) to the input x, where the parameters γ (gamma) and β (beta) are functions of an external condition vector c.

$$\mathbf{y} = \gamma(\mathbf{c}) \odot \mathbf{x} + \beta(\mathbf{c})$$

where:

- x is the input feature map (an intermediate activation in the denoiser network of shape).
- NOTE: this input (x) to FiLM is not Noisy 3D motion sequence (shape: n × q), but rather Intermediate features between multiple transformer blocks of Denoiser (say shape: n × d) where d is hidden layer's dimension.
- Condition: c = **[embed(t); encode(p)]** (a concatenation of the timestep embedding and the encoded 2D pose sequence).
- γ(c) and β(c) are functions that map the condition c to scale and shift vectors.
- ⊙ denotes the **Hadamard (element-wise) product**.

Flowchart:

- Input: Noisy 3D motion sequence (shape: n × q)
- Multiple transformer layers
- Intermediate feature: x (shape: n × d) // d is hidden dimension
- FiLM modulation here
- More transformer layers
- Output: Denoised 3D motion (shape: n × q)

Simple Linear Layer: A Static Transformation (γ = wx+b). The parameters W and b are **static**. They are learned during training and then remain fixed for every input during inference. They do not change based on the content of x or any external context.

1. **Dynamic vs. Static:** γ and β are not fixed model parameters. They are the outputs of a neural network (f_{film}) that takes c as input. This means the transformation applied to x changes dynamically based on the condition. $\gamma, \beta = f_{film}(\mathbf{c})$
2. **Feature-wise Modulation:** The operation γ ⊙ x + β is applied element-wise or channel-wise (*for efficiency: per-channel modulation*). If x is a feature vector of length D, then γ and β are also vectors of length D. This means each feature dimension in x can be individually scaled up, scaled down, or shifted. This is a much more fine-grained and expressive control mechanism than a full matrix multiplication.

Compute how the loss L depends on the FiLM parameters Θ_{film} :

$$\frac{\partial \mathcal{L}}{\partial \Theta_{\text{film}}} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{y}} \odot \mathbf{x}\right) \cdot \frac{\partial \gamma}{\partial \Theta_{\text{film}}} + \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \cdot \frac{\partial \beta}{\partial \Theta_{\text{film}}}$$

Dimensional Analysis: Let

- n = number of frames (S in paper)
- p = 2D pose dim per frame (e.g., 17 joints × 2 = 34),
- q = 3D pose dim per frame
  (e.g., 24 joints × 6D rotations + root pos + contact = 151)
- A = **embed(t)**, i.e., Timestep embedding
  and a = dim(A) (say ~128 or 256)
- B = **encode(n*p)**, i.e., encoded 2D poses.
  - Input: 2d poses of all frames (global video clip's 2D skeleton trajectory).
  - Goes through an **encoder network** (likely a temporal transformer or CNN). This encoder produces a **compact representation** of the entire sequence. Say b = dim(B).
  - Ouput: variable (often projected to same 256–512 range)
- Thus c = [A;B] => dim(c) = a+b.
- **Global Conditioning:** The same γ, β are applied to all frames because the condition represents global properties of the entire sequence
- **Efficiency:** If FiLM output n × d parameters, it would require:
  Input: (a + b) → Output: 2 × n × d
  This is computationally expensive and could overfit.

$$\begin{aligned} c &= [\,\text{embed}(t);\, \text{encode}(p)\,] && \in \mathbb{R}^{a+b} \\ (\gamma, \beta) &= \text{FiLM}(c) && \in \mathbb{R}^{2d} \\ h' &= \gamma \odot h + \beta && h, h' \in \mathbb{R}^{n \times d} \end{aligned}$$

Conditioned vs. Unconditioned Generation

- **Unconditioned Diffusion Model:** You ask the artist to "create something." They generate a random image from the static. It could be a cat, a dog, a car—whatever they learned during training.
- **Conditioned Diffusion Model:** You give the artist an **instruction**. You say, "Create a picture **conditioned on** the word 'cat'." Now, the artist uses that instruction (the "condition") to guide the denoising process, ensuring the final image is a cat.

In ViMo:

- **The Condition (c or p):** The sequence of 2D poses extracted from the input video.
- **The Goal:** Generate a 3D motion that matches the condition (the 2D poses).

What is "Classifier-Free Guidance"?

The Old, Complicated Way (Classifier Guidance):

Early diffusion models (e.g., guided diffusion in 2021) used an **external classifier** to guide generation. Imagine you have two people:

1. The **Artist (Denoiser)**: Knows how to denoise images.
2. The **Harsh Critic (Classifier)**: A separate AI that is an expert at identifying images. You show it a picture and it says, "That is 95% a cat."

To generate a "cat" image, you would start with noise and have the artist make a small step. Then you'd ask the critic, "Does this look more like a cat?" You'd use the critic's feedback to *push* the image in the "more cat-like" direction. This is powerful but complex, as it requires training two separate models.

The New, Simpler Way (Classifier-Free Guidance):
You *don't* train a separate classifier. Instead, you train a single diffusion model in two modes: You train only **one model**—the Artist—but you train it in two different modes:

- **Conditioned** *on the input signal (here: the 2D pose):* You show it a noisy image *and* the text "cat". It learns to denoise it into a cat.
- **Unconditioned** *(the conditioning is randomly dropped, e.g. replaced by a null token) during training:* You sometimes randomly **drop the condition**. You show it the same noisy image but with no text. It learns to denoise it into a random image.

So, the same network learns both "with conditioning" and "without conditioning."

During Training: They randomly drop the 2D pose condition 25% of the time. This trains the model to also work in an "unconditioned" mode.

During Generation: They use the "classifier-free guidance" formula. The final output is a blend: 75% of the "conditioned" output (faithful to the video) and 25% of the "unconditioned" output (creative, diverse). This balances fidelity (sticking to the video) and diversity (creating natural, varied motions).

At sampling time:

You can interpolate between these two outputs. If $D(m_t, t, p)$ is the denoiser conditioned on pose, and $D(m_t, t, \varnothing)$ is unconditioned, then guided prediction is:

$$\hat{m}_0 = (1 + w)\, D(m_t, t, p) - w\, D(m_t, t, \varnothing),$$

where w controls guidance strength.

- Larger w: more faithful to condition (pose), but less diversity (risk of overfitting).
- Smaller w: more diverse/random, but maybe less accurate.

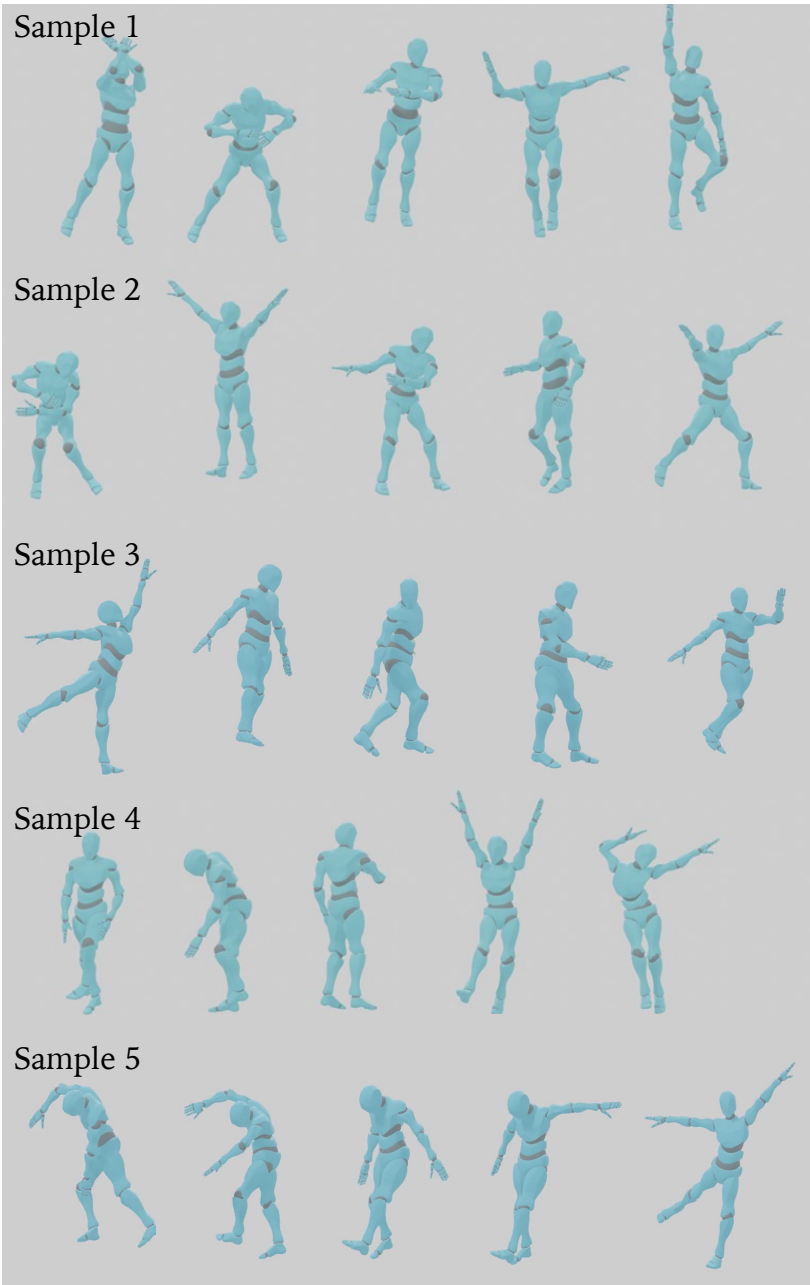

Fig. 3. Constructed Chinese classic dance dataset. ViMo can help to build high-quality motion data and utilize these data to benefit downstream tasks as our other applications illustrate.

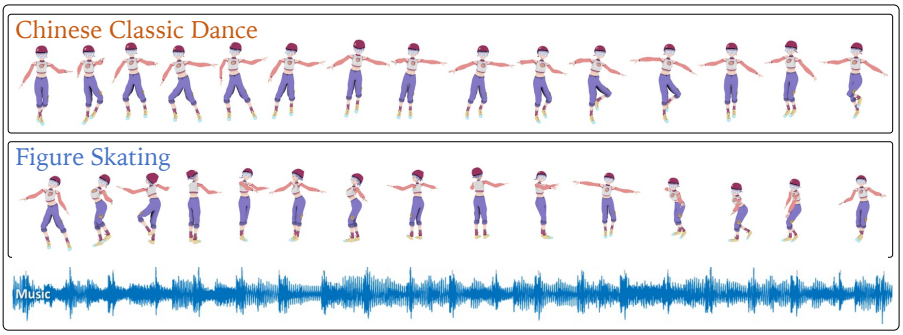

Fig. 4. Dance Motion Stylization given arbitrary music. Given a few references videos, our ViMo can conveniently extract motions and feed these data to music-to-dance models to learn motions in the corresponding style. This enables generate one style motion, kungfu style for instance, from a totally different kind of music such as pop music.

Loss Functions. The generated motions are required to be continuous, smooth and physically reasonable for practical use. Therefore, existing work has proposed a set of loss functions in the typical generation task to strengthen the quality of the produced motion. We follow the standard approach and take three of the most commonly used loss functions as auxiliaries to enhance the performance. The formulations of the three commonly used losses, which are 3D positions loss, velocities loss and foot contact loss respectively, are listed in the following:

$$\mathcal{L}_{\text{joints}} = \frac{1}{S}\sum_{i=1}^{S}\left\|FK\left(R_0^i\right) - FK\left(\hat{R}_0^i\right)\right\|_2^2, \tag{4}$$

$$\mathcal{L}_{\text{vel}} = \frac{1}{S-1}\sum_{i=1}^{S-1}\left\|\left(R_0^{i+1} - R_0^i\right) - \left(\hat{R}_0^{i+1} - \hat{R}_0^i\right)\right\|_2^2, \tag{5}$$

$$\mathcal{L}_{\text{foot}} = \frac{1}{S-1}\sum_{i=1}^{S-1}\left\|\left(FK\left(\hat{R}_0^{i+1}\right) - FK\left(\hat{R}_0^i\right)\right)\cdot \hat{f}_i\right\|_2^2. \tag{6}$$

The $FK(\cdot)$ denotes the forward kinematic function converting joint rotations into joint positions. Note that we use the model's own prediction $\hat{f}_i$ of the binary foot contact label's following [54]. The overall training loss is the adaptive weighted sum of the simple loss and the auxiliary loss functions:

$$\mathcal{L} = \underbrace{\mathcal{L}_{simple}}_{diffusion} + \underbrace{\lambda_1\mathcal{L}_{joints} + \lambda_2\mathcal{L}_{vel} + \lambda_3\mathcal{L}_{foot}}_{auxiliary} \tag{7}$$

The 6D Rotation Representation:

- **Joint Rotations (R) in 6D format:** This represents the orientation of each bone in the skeleton relative to its parent bone in the kinematic tree. The 6D representation is a continuous, non-redundant way to represent a 3D rotation, which is easier for neural networks to learn than traditional Euler angles (which suffer from gimbal lock) or Quaternions (which have a unit norm constraint). Mathematically, a 3D rotation matrix $\mathbf{R}_{3x3}$ can be approximated or represented by its first two column vectors, resulting in a 6D vector $[\mathbf{a}_1, \mathbf{a}_2]$. The third column can be recovered as $\mathbf{a}_3 = \mathbf{a}_1 \times \mathbf{a}_2$.
- **Root Position (root):** The global 3D translation (x, y, z) of the root joint (usually the pelvis). This defines the character's location in the world.
- **Foot Contact Labels (f):** A 4-dimensional binary vector indicating contact with the ground for the left foot, right foot, left toe, and right toe.

The 6D representation is a brilliant trick to avoid the problems of other 3D rotation parameterizations. Here's the step-by-step derivation.

Premise: A 3D rotation matrix $\mathbf{R} \in \mathbb{R}^{3\times3}$ is a **special orthogonal matrix**. This means:

1. $\mathbf{R}^T\mathbf{R} = \mathbf{I}$ (Orthonormal columns and rows).
2. $\det(\mathbf{R}) = +1$ (Proper rotation, no reflection).

Step 1: The Column Vector Interpretation
Any matrix can be defined by its column vectors. Let's define $\mathbf{R}$ as:

$\mathrm{R} = [a_1 \quad a_2 \quad a_3]$ where $\mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_3 \in \mathbb{R}^3$.

Step 2: Consequences of Orthonormality
From $\mathbf{R}^T\mathbf{R} = \mathbf{I}$, we get the constraints on the column vectors:

- $\| \mathbf{a}_1 \| = \| \mathbf{a}_2 \| = \| \mathbf{a}_3 \| = 1$ (Unit length).
- $\mathbf{a}_1 \cdot \mathbf{a}_2 = \mathbf{a}_1 \cdot \mathbf{a}_3 = \mathbf{a}_2 \cdot \mathbf{a}_3 = 0$ (Mutually orthogonal).

Step 3: The Key Insight for the 6D Representation
The first two columns, $\mathbf{a}_1$ and $\mathbf{a}_2$, are two orthogonal unit vectors in 3D space. They define a plane. The third column, $\mathbf{a}_3$, must be a unit vector orthogonal to this plane. There is only one such vector (up to sign, but the sign is fixed by the right-hand rule to ensure $\det(\mathbf{R}) = +1$).

Therefore, the third column is **uniquely determined** by the first two:

$\mathbf{a}_3 = \mathbf{a}_1 \times \mathbf{a}_2$, Proof**:** The cross product $\mathbf{a}_1 \times \mathbf{a}_2$ produces a vector that is:

1. Orthogonal to both $\mathbf{a}_1$ and $\mathbf{a}_2$ (satisfying the orthogonality constraint).
2. Has a magnitude of $\| \mathbf{a}_1 \| \| \mathbf{a}_2 \| \sin\theta = 1$ (since they are unit and orthogonal, $\sin 90^\circ = 1$), satisfying the unit length constraint.
3. Its direction follows the right-hand rule, ensuring a proper rotation matrix with $\det(\mathbf{R}) = +1$.

Conclusion: We do not need to predict all 9 elements of $\mathbf{R}$. We only need to predict the first two columns $(\mathbf{a}_1, \mathbf{a}_2)$, which form a 6D vector. The full, valid, orthonormal rotation matrix is then **reconstructed** without any approximation as: $\mathrm{R} = [a_1 \quad a_2 \quad a_1 \times a_2]$

Deconstructing the Loss Functions

The diffusion loss $\mathcal{L}_{\text{simple}}$ ensures the model learns to denoise the data. The auxiliary losses $\mathcal{L}_{\text{joints}}, \mathcal{L}_{\text{vel}}, \mathcal{L}_{\text{foot}}$ are applied to the model's final prediction of the *clean* motion at each denoising step, $\hat{m}_0 = D(m_t, t, p)$, to enforce physical plausibility and temporal coherence.

Let's define the notations clearly:

- S: The number of frames in the sequence.
- R_0^i: The **ground truth** joint rotations for frame i. The subscript '$_0$' denotes the "clean" data at diffusion step $t = 0$.
- $\hat{R}_0^i$: The **model's predicted** joint rotations for frame i (extracted from $\hat{m}_0$).
- $FK(\cdot)$: The **Forward Kinematics** function.
- $\hat{f}_i$: The model's predicted binary foot contact label for frame i.

3D Positions Loss ($\mathcal{L}_{\text{joints}}$)

Equation:

$$\mathcal{L}_{\text{joints}} = \frac{1}{S}\sum_{i=1}^{S} \| FK(R_0^i) - FK(\hat{R}_0^i) \|_2^2$$

What it does: This loss ensures the model's predicted skeleton has the same 3D shape as the ground truth skeleton.

The Forward Kinematics (FK) Function: This is the core of this loss. The FK function is a deterministic, differentiable function that takes the predicted joint rotations $\hat{R}$ and the root position, and computes the resulting 3D joint positions in global space.

- **How it works mathematically:** The skeleton is a kinematic tree. The position of a joint j is calculated by traversing the path from the root to j, sequentially applying the rotations of its parent joints.
  - Let the position of a joint j be $\mathbf{p}_j$.
  - Let its parent be $par(j)$, with a known bone vector $\mathbf{b}_j$ in the parent's local coordinate system.
  - The global position of j is: $\mathbf{p}_j = \mathbf{p}_{par(j)} + \mathbf{R}_{par(j)} \cdot \mathbf{b}_j$
- Here, $\mathbf{R}_{par(j)}$ is the *global* rotation matrix of the parent joint. This is found by recursively multiplying the local rotation matrices up the kinematic tree: $\mathbf{R}_j = \mathbf{R}_{par(j)} \cdot \mathbf{R}_j^{\text{local}}$, starting from the root.
- **Why it's necessary:** The model outputs rotations $\hat{R}$, but we ultimately care about the 3D mesh or joint positions. A small error in a root joint's rotation (e.g., the pelvis) can lead to a large error in the position of a leaf joint (e.g., the foot). $\mathcal{L}_{\text{joints}}$ directly penalizes these end-effector errors, which are perceptually most important.

[Angular] Velocities Loss ($\mathcal{L}_{\text{vel}}$)

Equation:

$$\mathcal{L}_{\text{vel}} = \frac{1}{S-1}\sum_{i=1}^{S-1} \| (R_0^{i+1} - R_0^i) - (\hat{R}_0^{i+1} - \hat{R}_0^i) \|_2^2$$

What it does: This loss enforces temporal smoothness and ensures the *rate of change* of the motion (its angular velocity) is correct.

Mathematical Interpretation:

- $(R_0^{i+1} - R_0^i)$: This is the finite difference approximation of the first derivative (ang. vel.) of the rotation param.s in the ground truth motion.
- $(\hat{R}_0^{i+1} - \hat{R}_0^i)$: This is the ang. vel. of the predicted rotation parameters.
- The L2 norm of their difference ensures that the model not only predicts correct poses per frame but also correct transitions between frames. This is critical for eliminating jitter and producing natural, smooth motion.

Rotation Parameters Difference as Angular Velocity: This is a discrete approximation of the time derivative. Let's formalize it.

Step 1: Define the Quantity
Let $\mathbf{r}(t)$ be the 6D rotation parameters of a specific joint as a continuous function of time t. The instantaneous rate of change (velocity in radians) of these parameters is the derivative $\frac{d\mathbf{r}(t)}{dt}$.

Step 2: Discrete Approximation
In a discrete-time system (like our motion sequence with frames at times $t_1, t_2, \ldots, t_S$), we approximate the derivative using finite differences. The forward difference approximation at frame i is:

$$\frac{d\mathbf{r}(t_i)}{dt} \approx \frac{\mathbf{r}(t_{i+1}) - \mathbf{r}(t_i)}{\Delta t}$$

where Δt = time interval between frames (e.g., $\Delta t = 1/30$ sec for 30 FPS).

Step 3: Simplification for Loss Function
In the loss function $\mathcal{L}_{\text{vel}}$, we are comparing the *difference* between the predicted and ground-truth motions. Since Δt is a constant positive scalar across all sequences, it can be factored out and absorbed into the loss weight λ_2. Therefore, minimizing the difference in the unnormalized finite differences $(\mathbf{r}_{i+1} - \mathbf{r}_i)$ is **equivalent** to minimizing the difference in the approximated velocities.

Mathematically:

$$\mathcal{L}_{\text{vel}} \propto \sum \left\| \left(\frac{d\mathbf{r}}{dt}\right)^{\text{gt}} - \left(\frac{d\mathbf{r}}{dt}\right)^{\text{pred}} \right\|^2 \propto \sum \left\| (\mathbf{r}_{i+1} - \mathbf{r}_i)^{\text{gt}} - (\mathbf{r}_{i+1} - \mathbf{r}_i)^{\text{pred}} \right\|^2$$

This loss ensures that the *change* in the joint's rotational state between frames is correct, which is a direct measure of its **angular velocity** in the parameter space.

Foot Contact Loss ($\mathcal{L}_{\text{foot}}$)

Equation:

$$\mathcal{L}_{\text{foot}} = \frac{1}{S-1} \sum_{i=1}^{S-1} \| (FK(\hat{R}_0^{i+1}) - FK(\hat{R}_0^{i})) \cdot \hat{f}_i \|_2^2$$

What it does: This is a **physics-based loss** designed to eliminate "foot sliding," a common artifact in generated motions where the feet slide unnaturally on the ground during steps.

Mathematical Deconstruction:

1. $FK(\hat{R}_0^{i+1}) - FK(\hat{R}_0^{i})$: This is the 3D displacement (velocity vector) of all joints between frame i and $i+1$, calculated from the predicted poses. Let's call this $\mathbf{v}_{\text{joints}}^i$, a matrix where each row is the velocity vector for a specific joint.
2. $\hat{f}_i$: This is the model's own prediction of the binary foot contact label for frame i. It's a 4D vector, e.g., $[1,0,1,0]$ meaning the left foot and left toe are in contact with the ground, while the right foot and toe are not.
3. **The Hadamard Product** $\cdot$: This is an element-wise multiplication. The operation $\mathbf{v}_{\text{joints}}^i \cdot \hat{f}_i$ is a bit tricky from a dimensionality perspective. In practice, this loss is applied **only to the foot and toe joints**. The vector $\hat{f}_i$ is used to mask the velocities of these specific joints.
   - ✓ If a foot is predicted to be in contact ($\hat{f}_i = 1$), its velocity should be close to zero. The loss penalizes any non-zero velocity for that joint.
   - ✓ If a foot is predicted to be *not* in contact ($\hat{f}_i = 0$), its velocity is unpenalized by this loss, as it is free to move.

Definitions:

- Let J be the total number of joints (e.g., 24 in their 3D skeleton).
- The FK function outputs a tensor of 3D positions for all joints: $\mathbf{P}^i \in \mathbb{R}^{J\times 3}$ for frame i.
- The velocity $\mathbf{V}^i$ is computed as $\mathbf{V}^i = \mathbf{P}^{i+1} - \mathbf{P}^i$. This is also a tensor in $\mathbb{R}^{J\times 3}$.

The Masking Operation:
The model predicts a foot contact vector $\hat{\mathbf{f}}_i \in \mathbb{R}^4$. We need to apply this to the relevant joints. We construct a **sparse mask vector** $\mathbf{m}_i \in \mathbb{R}^J$.

- Let $\mathcal{J}_{\text{feet}} = \{j_1, j_2, j_3, j_4\}$ be the set of indices corresponding to the left foot, right foot, left toe, and right toe joints.
- We set $\mathbf{m}_i[j] = \hat{\mathbf{f}}_i[k]$ if joint j is the k-th foot joint (where $k = 1,2,3,4$).
- For all other joints $j \notin \mathcal{J}_{\text{feet}}$, we set $\mathbf{m}_i[j] = 0$.

Now, to apply the mask to the velocity tensor $\mathbf{V}^i \in \mathbb{R}^{J\times 3}$, we perform an element-wise multiplication (**Hadamard product**) along the joint dimension, broadcasting the mask across the 3 spatial dimensions.

$$\mathbf{V}_{\text{masked}}^i = \mathbf{m}_i \odot \mathbf{V}^i$$

where $\odot$ denotes broadcasting $\mathbf{m}_i$ from shape (J) to $(J, 3)$ and then element-wise multiplication.

Result: The loss $\mathcal{L}_{\text{foot}} = \frac{1}{S-1}\sum_i \| \mathbf{V}_{\text{masked}}^i \|^2$ is simply the sum of squared velocities, but **only for the foot joints that the model itself has predicted to be in contact**. If the model thinks a foot is on the ground ($\hat{f} \approx 1$) but it's actually sliding (V is large), this loss will be high.

Foot Contact Label Prediction:

Traditional vs. ViMo's Self-Supervised Method

This phrase *"model's own prediction $\hat{f}_i$ of the binary foot contact label's following [54]"*means ViMo adopts the **self-supervised** strategy for foot contact labeling from the EDGE paper [54]. Instead of using pre-defined, fixed foot contact labels, the model *learns to predict* them on-the-fly, and these predictions are used in the loss.

Traditional Method (Pre-defined, Fixed Labels):
In a traditional, supervised pipeline, you would have a separate, pre-processing step to **define** what "foot contact" means.

1. You take your ground-truth 3D motion data (from MoCap).
2. You apply a **fixed heuristic** to compute the labels. A common heuristic is: $f_i^{\text{gt}} = \mathbb{I}(\| \mathbf{v}_{\text{foot},i} \|_2 < \epsilon)$

where $\mathbb{I}$ is the indicator function, $\mathbf{v}_{\text{foot},i}$ is the velocity of the foot joint computed from the *ground-truth* positions, and ϵ is a manually set threshold (e.g., 5 cm/s).

3. These labels f^{gt} are now **fixed ground-truth data**. You would then train a model (or a separate branch of your main model) to predict $\hat{f}_i$ using a standard binary classification loss (e.g., Binary Cross-Entropy) against these fixed labels.

Problems: This method is entirely dependent on the heuristic. The threshold ϵ is arbitrary and may not be optimal. It doesn't allow the model to learn what "contact" means in a way that is most useful for generating physically plausible motion.

ViMo's Self-Supervised Method:
ViMo integrates the contact label prediction directly into the motion generation objective **without fixed labels**.

1. **Architecture:** The model has a single output head that predicts the entire 151-dimensional vector per frame. This vector is decomposed into rotations (144D), root position (3D), and **foot contact labels (4D)**. There is no separate model; it's all part of the same denoising network $D(m_t, t, p)$.
2. **Training Signal:** The model receives **no direct supervision** (like BCE loss) for $\hat{f}_i$. Instead, the only signal for $\hat{f}_i$ comes from the gradient of the $\mathcal{L}_{\text{foot}}$ loss. **The Learning Dynamics:**
   - ➢ The model's goal is to minimize the total loss $\mathcal{L}$.
   - ➢ The $\mathcal{L}_{\text{foot}}$ term is $\| (\mathbf{V}^i) \cdot \hat{f}_i \|^2$.
   - ➢ The model has two ways to reduce this loss:
     a) **Adjust the Pose:** Change the predicted rotations $\hat{R}$ so that the foot velocity $\mathbf{V}^i$ becomes near zero for frames where contact is expected.
     b) **Adjust the Contact Label:** Change the predicted $\hat{f}_i$ to be close to 0 for frames where the foot velocity is high (so that it doesn't get penalized).
   - ➢ Through training, the model **discovers a correlation**: to minimize the loss, it should predict $\hat{f}_i \approx 1$ precisely when the foot velocity is low, and $\hat{f}_i \approx 0$ when it is high. It effectively **learns the heuristic itself** in a way that is perfectly tailored to its own motion predictions.

The traditional method *imposes* a definition of contact. ViMo's method *encourages the model to find a self-consistent definition* that results in non-sliding feet.

The Innovation: The model is trained to predict $\hat{f}_i$ directly as part of its output. The foot contact loss $\mathcal{L}_{\text{foot}}$ then uses this *prediction* to penalize sliding. This creates a feedback loop: if the model predicts a foot is on the ground ($\hat{f}_i \approx 1$) but the FK-calculated velocity is high, the loss will be large, forcing the model to adjust either the pose (to reduce velocity) or the contact label. This iterative refinement leads to physically plausible foot-ground interaction without needing explicit, hard-coded labels.

3.4 Applications

To highlight the importance and potential of the proposed new video-to-motion generation method, we demonstrate how it elevates three widely used motion applications specifically in this section.

3D Motion Dataset Construction via Diffusion: The motion dataset -

- Human 3.6m [22],
- Finedance: Fine-grained Choreography Dataset [24],
- AMASS: Archive of Motion Capture As Surface Shapes [32],
- The KIT Motion-Language Dataset [43],
- Action2Motion: Conditioned Generation of 3D Human Motions [33]
- KIT Whole-Body Human Motion Database [15]

is the cornerstone for motion-related tasks since it is essential to power up model training, feature learning and performance evaluation. However, it is time-consuming to build up a motion dataset compared with other modalities such as images or videos – Mocap systems could be million level and some actions need to be performed by human actors in the studio. Moreover, some desirable motions like skiing, water sports or celebrity dancing are almost impossible to collect due to the difficulties of recording outdoor activities as well as recruiting professional actors. Hence, it is meaningful to design an alternative approach to construct a Motion dataset in a cost-effective and efficient manner.

Dance data is markedly deficient in existing available motion data where Chinese classic dance is conspicuously absent from the repertoire. Hence, we first collect 52 videos from the Chinese annual dancing competition, which are all solo-dance, continuous and high-quality. Then the 2D poses are extracted by AlphaPose [13] and the confidence of joints in missing frames is filled with zero. A simple interpolation and completion process is employed to revise the 2D pose sequences to be continuous and complete. Finally, we generated 3D motions using *ViMo* from all these video inputs to amplify the numbers, and carefully

chosen 750 clips. This dataset spans a total time length of up to 63 minutes. Our generated Chinese Classic motion dancing samples are illustrated in Fig 3. It can be seen that our ViMo can create realistic and various motions. This motion dataset expands the dance genres and could improve performance for many tasks such as music-to-motion generation, motion recognition and motion prediction. Details are dicussed in the experiment section.

Few-shot Dancing Stylization from Videos The existing approach to music-to-dance generation [54] restricts the dance generation to 1-25 predefined categories. Our model significantly overcomes this limitation by generating dances in any style using just a few reference videos. In essence, this application takes videos and music of any length as input and generates motions that adopt the pose style from the video and the beat alignment from the music. This principle can also be applied to audio-based and text-based motion generation pipelines.

We demonstrate a successful example using several figure skating videos as references. To accomplish this, we integrated zero-convolution blocks, similar to those in the basic music-to-dance diffusion baseline [54]. The efficiency of these zero-convolution blocks is demonstrated in experiments results. We then retrained this model using prompts from the source figure skating video. By learning the distribution of video-guided data, our model gains the ability to synthesize dance movements similar to figure skating. Examples of the generated dances are presented in Fig 4, confirming the potential of our ViMo to extract styles and translate them between different modalities.

Video-guided Motion Completion Motion Completion [16,12] plays an important role in animation and 3D-featured film production. Users only need to provide some keyframes or clips as guidance to obtain a complete movement, which significantly improves users' effort and efficiency. Based on the type of missing parts, motion completion is usually categorized into three types, in-between, in-filling and blending [12].

Users can perform motion in-betweening by providing a reference motion c_{const} with the same range of $m \in \mathbb{R}^{S\times 151}$ and a mask $b \in \{0,1\}^{N\times 151}$, where $\boldsymbol{m}$ is all 1 's in the first and last n frames and 0 everywhere else. This would result in a sequence N frames long, where the first and last n frames are provided by the reference motion and the rest is filled in with a plausible "in-between" dance that smoothly connects the constraint frames.

Inspired by the image inpainting process [30], the proposed diffusion model **ViMo** could be leveraged with standard masked denoising techniques to perform the motion completion. A simple example of tackling the motion in-between case, our model could expand a distribution space by the input videos to fill in the missing frames in the motion. Mathematically, given joint-wise positions indicated by a binary mask b, we perform the following at every denoising time step:

$$\hat{\mathbf{m}}_{t-1} := \boldsymbol{b} \odot q\left(c_{const}, t-1\right) + (1-\boldsymbol{b}) \odot \hat{\mathbf{m}}_{t-1} \tag{8}$$

What are Zero-Convolution Blocks?

Zero-convolution blocks are a **parameter-efficient fine-tuning technique** used to adapt pre-trained models to new tasks or domains without catastrophic forgetting of the original knowledge.

Mathematical Foundation:

1. **Start with a pre-trained weight matrix** $W_{\text{pretrained}} \in \mathbb{R}^{m \times n}$
2. **Create a new trainable weight matrix** $W_{\text{new}} \in \mathbb{R}^{m \times n}$ initialized to **zeros**
3. **During forward pass**, the effective weights become:

$$W_{\text{effective}} = W_{\text{pretrained}} + W_{\text{new}}$$

4. **At initialization**: Since $W_{\text{new}} = \mathbf{0}$, the network behaves exactly like the original pre-trained model:

$$W_{\text{effective}} = W_{\text{pretrained}} + \mathbf{0} = W_{\text{pretrained}}$$

5. **During fine-tuning**: Only W_{new} is updated, while $W_{\text{pretrained}}$ remains frozen

Why This is Brilliant for ViMo's Few-Shot Stylization

In ViMo's context, they're using EDGE as a pre-trained music-to-dance model and want to adapt it to new dance styles with very few examples.

Traditional Fine-Tuning Problem:

- If you fine-tune all parameters, you risk **catastrophic forgetting** - the model forgets how to align with music while learning the new style
- With few examples, overfitting is severe

Zero-Convolution Solution:

1. **Keep the original EDGE model frozen** - preserves all music-dance alignment knowledge
2. **Add parallel zero-initialized layers** that learn to modify the output toward the target style
3. **At start**: Output = Original EDGE output (perfect music alignment preserved)
4. **During training**: Zero-convolutions gradually learn to "shift" the output toward the target style

Architecture in ViMo

From the ViMo paper description:

- "The zero-convolution module consists of new trainable networks with the same structure and a zero-convolution network attached and concatenated with the original freezing models"

This suggests they're using a **residual connection** approach:

$$\text{Output} = f_{\text{frozen}}(\text{input}) + f_{\text{zero-conv}}(\text{input})$$

Where:

- f_{frozen} = original EDGE layers (frozen)
- $f_{\text{zero-conv}}$ = new layers initialized to zeros

where $\odot$ is the element-wise product, replacing the known regions with forward-diffused samples of the constraint. This technique allows editability for motion generation in referenced styles. Visualization results are demonstrated in Fig 6.

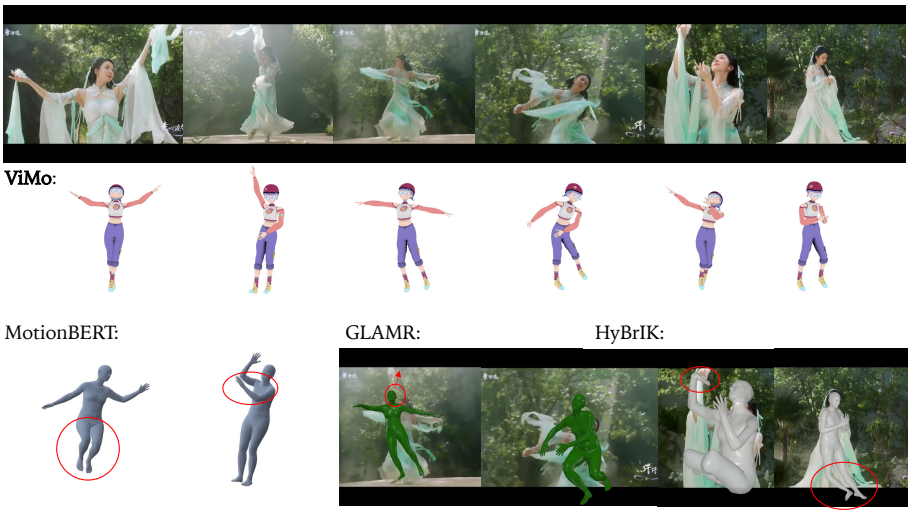

Fig. 5. Qualitative motion results with different methods. It can be seen that proposed ViMo generate relatively more plausible motions compared with other methods.

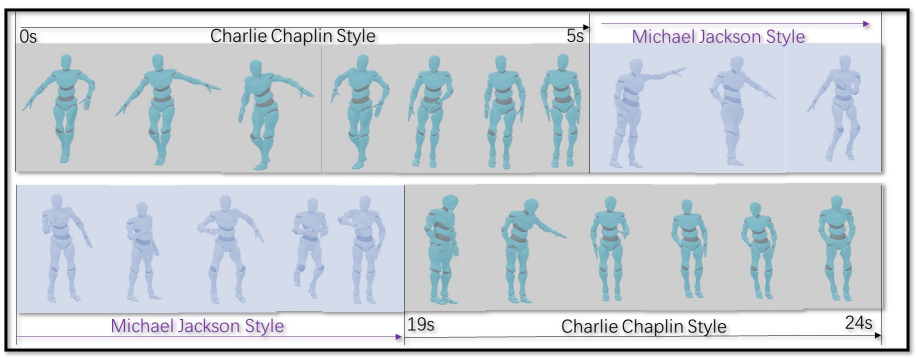

Fig. 6. Completion example of inpainting a Charlie Chaplin style motion with a Michael Jackson dancing style clip. ViMo provides a convenient way to manipulate motion styles and generate natural and smooth motion results.

4 Experiments

To thoroughly assess the effectiveness of the proposed method, we have reported comprehensive evaluation metrics, alongside the outcomes of user study in the following paragraphs.

4.1 Configurations

Dataset. In this work, AIST++ [25] dataset is used, which is consisting of 1,408 high-quality dance motions paired with music from 10 kinds of genres. Each 3D motion inside corresponds to 2D videos from 8 angles. Among 10 dances genres in AIST++, we used the two genres – break and middle hip-hop for testing and the rest 8 genres for training. All training examples are cut to 5 seconds, 30 FPS. During training, the multi-view projected 2D pose sequences from different camera angles are used as input data to enable our ViMo model has the ability to reconstruct 3D skeletons from different views.

In addition, as an important application of our ViMo, a 3D motion dataset of Chinese classic dancing via diffusion is built. This dataset contains 52 Chinese classic dancing videos together with 750 generated 3D dancing motions. This dataset would benefit tasks on generation, prediction and recognition for more study on motion and we will release the dataset. More details can be found in Section 4.2.1.

Baseline and Backbone Methods. We have selected two advanced state-of-the-art methods as baseline and backbone methods.

First, we employ MotionBERT [71], which has recently drawn attention for its superior ability to reconstruct motion from videos. To conduct a thorough assessment of its proficiency in deriving 3D motions from casual videos, we re-train MotionBERT on the comprehensive AIST++ dataset to conduct a fair comparison.

Second, we incorporate EDGE [54], a state-of-the-art music-to-dance conversion model employing diffusion-based methods for efficient few-shot stylization from videos, and characterized by its transformer-based diffusion framework as our backbone methods in Section 4.2.3. We test our model using a diverse range of dance videos, including Chinese classical dance, figure skating, and dances emulating Michael Jackson's style, to generate movements within these distinct categories.

The evaluation emphasize the quality, variety, and musical synchronization of our dance generation, using the baseline methodologies as reference points. For an in-depth analysis of the metrics used, please see the following paragraph.

Metrics. Our evaluation of the generated dances encompasses three critical dimensions: the quality of the dances, the range of motion diversity, and the synchronicity between the music rhythms and the generated movements.

Specifically, for assessing motion quality, we employ the Frechet Inception Distances (FID) [17] and the Physical Foot Contact Score [54]. These metrics are calculated by comparing the generated dances against all motion sequences (inclusive of both training and test data) from the AIST++ dataset, focusing on kinetic features [38] (labelled as 'FID_k') and handpicked geometric features [37] (labelled as 'FID_m'), both of which are extracted using the toolkit from [14].

For motion diversity, we compute the mean distance between features of generated movements, adhering to the method outlined in [25], denoted as 'Div_k; and 'Div_m' for kinetic features and geometric features respectively.

To quantify the alignment of music and movements, we measure the average temporal discrepancy between each musical beat and its nearest corresponding dance beat, denoting this as the Beat Align Score (BA).

Additionally, for qualitative assessments, we conducted user studies involving 44 uniquely generated 3D motions on 'Win Rate', which is the proportion of the preferred choice among candidates and indicates the winners' relevance and applicability in real-world scenarios.

Implementation Details. We set the sequence to 5 seconds and aligned all the videos and motions with fps to 30. The target motion contains the 6D rotation extracted from the SMPL θ parameter and the global position of the root joint. The binary foot contact labels $\{0,1\}^{N\times 4}$ are pre-computed from whether foot velocities of both foot joints and toe joints along the up-axis are near zero. The sampling diffusion step T is set to be 1000. We adopt [49] to accelerate the sampling process by predicting the posterior distribution with 50 steps for each DDIM sampling. The learning rate is set to be 1e-4, the total training epoch is 1000 and the optimizer is ADAN [58] with a decay rate to be 0.02.

4.2 Evaluation and Application Results.

Comparison between ViMo and MotionBERT We evaluated the motion-extracting capabilities of the proposed ViMo against MotionBERT [71] on dataset AIST++ [25].

The MotionBERT models provide two different ways to obtain human motions: 3D pose estimation (in halpe-26 skeletons) and human mesh recovery (SMPL shape and pose). We choose the human mesh recovery model to align the SMPL pose format with ViMo and simply the method to predict the SMPL pose only and retrain their model on the AIST++.

As shown in Table 1, the left part demonstrates the quantative result on motion quality (FID) and diversity (Div). It demonstrates that our model outperforms MotionBERT [71] in terms of the FID index and Diversity Index, indicating higher quality and diversity. However, as discussed in the literature [54], these indices are not always reliable. Indeed, there can be instances where perceptual realism decreases even as these indices increase. Qualitative comparison are shown in Fig 5.

Table 1. Performance comparison with MotionBERT [71] and ViMo. Experiments are evaluated on two dance genres of AIST++ [25] for testing. The subscribe k and m denote the metric of kinetic and manually selected geometry features respectively. The results shows the superiority of our proposed ViMo on quantitative scores and user study.

Model	$FID\_k \downarrow$	$FID\_m \downarrow$	$Div\_k \uparrow$	$Div\_m \uparrow$	Win Rate↑
MotionBERT	19.22	9.03	**5.27**	5.02	23.95%
ViMo	**15.70**	**8.25**	4.85	**5.77**	**76.05%**

To substantiate these findings, we conduct a user study and report metrics as shown in the right part of Table 1. In this study, we create a comprehensive questionnaire disseminated through Wenjuanxing, an online platform, to recruit participants. A total of 102 participants are enlisted to compare the performance of *ViMo* and MotionBERT. We present 44 video demonstrations and asked 22 questions in the questionnaire. Participants are asked to choose which model performed better or to rate their satisfaction with the motion generation quality. The participants also involves the user study in our stylization experiments in Section 4.2.3.

Furthermore, we curate a small set of extremely challenging casual videos featuring renowned Chinese dancers and advertising videos sourced directly from the internet. These videos, characterized by continuous action but complex camera movements and editing, are representative of our target casual videos. Visual inspection of the results revealed that MotionBERT struggled with this dataset, generating unusual actions. In contrast, our model proved robust, managing to produce a dance sequence that closely resembled the original and maintained the same rhythm. It is evident that the proposed ViMo can generate smoother and more plausible motions across all view inputs. Although MotionBERT sometimes could produce more accurate results on specific frames, it exhibited a tendency towards excessive shaking and jittering. It is noteworthy that the AIST++ test dataset, which does not include many camera movements, still posed a challenge to MotionBERT's reconstruction abilities.

3D Motion Dataset via Diffusion In collaboration with *ViMo*, we successfully synthesize an extensive collection of 750 Chinese classic dance motion sequences, as illustrated in Fig 3. These sequences are remarkably faithful to the original dance videos, capturing the essence and fluidity of Classic dance. We divide each extended video into segments of 5 seconds, creating concise motion clips. This segmentation significantly streamline the training process for our music-to-motion synthesis tasks. To assess the practicality of this dataset, we utilize it as a foundational training set for a music-to-dance generation task. The outcomes were impressive, showcasing an increased variety in dance and the generation of authentic Chinese classic dance styles that seamlessly synchronize with music of varying lengths. Remarkably, *ViMo* possesses the potential to expand this dataset by 5 or even 50 times, paving the way for large-scale

data generation from limited video resources. This methodology holds promise for application across diverse video categories.

Few-shot Dancing Stylization from Videos The task few-shot dancing stylization focusing on generate 'music-aligned' motion in this style given these limited videos. Our approach involve the analysis of merely two figure skating videos, each sourced from Olympic competitions. The dynamic nature of figure skating, with its unique movement patterns distinct from Chinese classic dance forms, presented a challenge. These patterns often involve repetitive and iconic sequences. Due to the sport's dynamism, the camera keeps motion to make the skater remained the focal point.

To obtain this task, we first build a small dataset with figure skating data samples paired with music from the videos as discussed in the previous Section. Next, the backbone method EDGE [54], which is the state-of-the-art music-to-dance diffusion-based method, is adopted to align the generated motion given arbitrary music. We find that directly finetuning the EDGE model with new data samples works basically, except for the learned style influenced by EDGE training data styles. This influence makes the generated motions partially consist of the street dances style in AIST++.

To improve this, we design a zero convolution module to make the generated motions focus more on the distribution in the expected styles in the few-shot videos while keeping benefits in the well-trained parameters from EDGE checkpoints. Specifically, the zero convolution module consists of new trainable networks with the same structure and a zero convolution network attached and concatenated with the original freezing models [66]. The newly attached structure is initialized by the original checkpoints but can be trained to learn the new style distributions while the freezing parts provide the learned music-to-dance ability in the pre-trained backbone checkpoints.

To demonstrate the merits of the zero convolution design for our few-shot dancing stylization, we test our models using a diverse range of dance videos, including Chinese classical dance, figure skating, and dances emulating Michael Jackson's style (labelled as Type-1,2,3 respectively), to generate movements within these distinct categories as shown in Table 2. It can be seen that our proposed design labelled by 'ZeroConv' shows superiority especially for the 'Win Rate' in the user study.

Video-guided Motion Completion We aim to showcase our model's proficiency in accomplishing tasks such as completing interrupted motions, motion in-between, and seamlessly merging movements. We selected a small collection of casual videos in specific styles, such as depicting characters like Michael Jackson and Charlie Chaplin, to demonstrate this capability. Given expected condition control in different timestamp, ViMo can generate natural motions and smoothly switching between these styles . The results are shown in Fig 6. The reconstructed motions not only include significant characteristics from the previous style but also show a high degree of fidelity in target motion style. These results illustrate

Table 2. Evaluation on motion generation metrics for the proposed design. The 'ZeroConv' not only approximately outperforms the directly fintuning in all the quantitatively scores in BA, PFC and diversity, but also notably preferred for the 'Win Rate' in our user study. Note that each type is denoted in a distinct style category.

Type-1	BA ↑	PFC ↓	*Div_k* ↑	*Div_m* ↑	Win Rate ↑
ViMo+ZeroConv	**0.243**	**1.49**	4.32	4.59	**56.27**%
ViMo+finetune	0.239	1.59	**4.42**	**4.80**	43.73%
Type-2	BA ↑	PFC ↓	*Div_k* ↑	*Div_m* ↑	Win Rate ↑
ViMo+ZeroConv	**0.236**	1.50	**4.51**	4.32	**65.89**%
ViMo+finetune	0.226	**1.44**	4.20	**4.51**	34.11%
Type-3	BA ↑	PFC ↓	*Div_k* ↑	*Div_m* ↑	Win Rate ↑
ViMo+ZeroConv	0.239	**1.47**	**4.78**	**4.89**	**57.65**%
ViMo+finetune	**0.244**	1.52	4.70	4.73	43.35%

ViMo's ability to control motions in expected style, offering users an intuitive approach to motion editing and semantic control.

5 Conclusion

In this paper, we innovate a new question to generate multiple possible 3D motions from the casual video which films complicated actions with varied camera positions and complex editing. To address this problem, we reveal a novel transformer-diffusion-based framework which could leverage the 2D pose sequences to similar 3D motions without explicitly estimating the camera angles. Visualization comparison results show that our method could outperform the existing work dramatically. Experimental results show that our method could empower three crucial motion applications, thus offering remarkable value for academics and industry. Our work is the first attempt toward this goal and we believe it will open new opportunities and encourage future research to exploit video for more complex motion generation, achieving more semantic, scene-aware and multi-person stages. Striking breakthroughs and tremendous benefits could be expected in the future with the exploration of this direction.

References

1. Aberman, K., Li, P., Lischinski, D., Sorkine-Hornung, O., Cohen-Or, D., Chen, B.: Skeleton-aware networks for deep motion retargeting. Transactions on Graphics (2020)
2. Alexanderson, S., Nagy, R., Beskow, J., Henter, G.E.: Listen, denoise, action! audio-driven motion synthesis with diffusion models. ACM Transactions on Graphics (TOG) **42**(4), 1–20 (2023)
3. Aristidou, A., Yiannakidis, A., Aberman, K., Cohen-Or, D., Shamir, A., Chrysanthou, Y.: Rhythm is a dancer: Music-driven motion synthesis with global structure. arXiv:2111.12159 (2021)

4. Cai, Y., Ge, L., Liu, J., Cai, J., Cham, T.J., Yuan, J., Thalmann, N.M.: Exploiting spatial-temporal relationships for 3d pose estimation via graph convolutional networks. In: Proceedings of the IEEE/CVF international conference on computer vision. pp. 2272–2281 (2019)
5. Cai, Z., Yin, W., Zeng, A., Wei, C., Sun, Q., Wang, Y., Pang, H.E., Mei, H., Zhang, M., Zhang, L., et al.: Smpler-x: Scaling up expressive human pose and shape estimation. arXiv preprint arXiv:2309.17448 (2023)
6. Ceylan, D., Huang, C.H.P., Mitra, N.J.: Pix2video: Video editing using image diffusion. In: Proceedings of the IEEE/CVF International Conference on Computer Vision. pp. 23206–23217 (2023)
7. Chen, W., Jiang, Z., Guo, H., Ni, X.: Fall detection based on key points of human-skeleton using openpose. Symmetry **12**(5), 744 (2020)
8. Cheng, Y., Yang, B., Wang, B., Tan, R.T.: 3d human pose estimation using spatio-temporal networks with explicit occlusion training. In: Proceedings of the AAAI Conference on Artificial Intelligence. vol. 34, pp. 10631–10638 (2020)
9. Ci, H., Wang, C., Ma, X., Wang, Y.: Optimizing network structure for 3d human pose estimation. In: Proceedings of the IEEE/CVF international conference on computer vision. pp. 2262–2271 (2019)
10. Ci, H., Wu, M., Zhu, W., Ma, X., Dong, H., Zhong, F., Wang, Y.: Gfpose: Learning 3d human pose prior with gradient fields. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 4800–4810 (2023)
11. Degardin, B., Neves, J., Lopes, V., Brito, J., Yaghoubi, E., Proença, H.: Generative adversarial graph convolutional networks for human action synthesis. In: WACV. pp. 1150–1159 (2022)
12. Duan, Y., Shi, T., Zou, Z., Lin, Y., Qian, Z., Zhang, B., Yuan, Y.: Single-shot motion completion with transformer. arXiv preprint arXiv:2103.00776 (2021)
13. Fang, H.S., Li, J., Tang, H., Xu, C., Zhu, H., Xiu, Y., Li, Y.L., Lu, C.: Alphapose: Whole-body regional multi-person pose estimation and tracking in real-time. IEEE Transactions on Pattern Analysis and Machine Intelligence (2022)
14. Gopinath, D., Won, J.: Fairmotion-tools to load, process and visualize motion capture data (2020)
15. Guo, C., Zuo, X., Wang, S., Zou, S., Sun, Q., Deng, A., Gong, M., Cheng, L.: Action2motion: Conditioned generation of 3d human motions. In: Proceedings of the 28th ACM International Conference on Multimedia. pp. 2021–2029 (2020)
16. Harvey, F.G., Yurick, M., Nowrouzezahrai, D., Pal, C.: Robust motion in-betweening. ACM Transactions on Graphics (TOG) **39**(4), 60–1 (2020)
17. Heusel, M., Ramsauer, H., Unterthiner, T., Nessler, B., Hochreiter, S.: Gans trained by a two time-scale update rule converge to a local nash equilibrium. Advances in neural information processing systems **30** (2017)
18. Ho, J., Jain, A., Abbeel, P.: Denoising diffusion probabilistic models. Advances in neural information processing systems **33**, 6840–6851 (2020)
19. Holmquist, K., Wandt, B.: Diffpose: Multi-hypothesis human pose estimation using diffusion models. In: Proceedings of the IEEE/CVF International Conference on Computer Vision. pp. 15977–15987 (2023)
20. Hong, F., Zhang, M., Pan, L., Cai, Z., Yang, L., Liu, Z.: Avatarclip: Zero-shot text-driven generation and animation of 3d avatars. arXiv preprint arXiv:2205.08535 (2022)
21. Huang, S., Wang, Z., Li, P., Jia, B., Liu, T., Zhu, Y., Liang, W., Zhu, S.C.: Diffusion-based generation, optimization, and planning in 3d scenes. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 16750–16761 (2023)

22. Ionescu, C., Papava, D., Olaru, V., Sminchisescu, C.: Human3. 6m: Large scale datasets and predictive methods for 3d human sensing in natural environments. IEEE transactions on pattern analysis and machine intelligence **36**(7), 1325–1339 (2013)
23. Li, B., Zhao, Y., Zhelun, S., Sheng, L.: Danceformer: Music conditioned 3d dance generation with parametric motion transformer. In: AAAI (2022)
24. Li, R., Zhao, J., Zhang, Y., Su, M., Ren, Z., Zhang, H., Tang, Y., Li, X.: Finedance: A fine-grained choreography dataset for 3d full body dance generation. In: Proceedings of the IEEE/CVF International Conference on Computer Vision. pp. 10234–10243 (2023)
25. Li, R., Yang, S., Ross, D.A., Kanazawa, A.: Ai choreographer: Music conditioned 3d dance generation with aist++. In: Proceedings of the IEEE/CVF International Conference on Computer Vision. pp. 13401–13412 (2021)
26. Li, W., Liu, H., Tang, H., Wang, P., Van Gool, L.: Mhformer: Multi-hypothesis transformer for 3d human pose estimation. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 13147–13156 (2022)
27. Li, X., Thickstun, J., Gulrajani, I., Liang, P.S., Hashimoto, T.B.: Diffusion-lm improves controllable text generation. Advances in Neural Information Processing Systems **35**, 4328–4343 (2022)
28. Lin, T.Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., Zitnick, C.L.: Microsoft coco: Common objects in context. In: Computer Vision–ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6-12, 2014, Proceedings, Part V 13. pp. 740–755. Springer (2014)
29. Liu, X., Wu, Q., Zhou, H., Du, Y., Wu, W., Lin, D., Liu, Z.: Audio-driven co-speech gesture video generation. Advances in Neural Information Processing Systems **35**, 21386–21399 (2022)
30. Lugmayr, A., Danelljan, M., Romero, A., Yu, F., Timofte, R., Van Gool, L.: Repaint: Inpainting using denoising diffusion probabilistic models. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 11461–11471 (2022)
31. Maheshwari, S., Gupta, D., Sarvadevabhatla, R.K.: Mugl: Large scale multi person conditional action generation with locomotion. In: WACV (2022)
32. Mahmood, N., Ghorbani, N., Troje, N.F., Pons-Moll, G., Black, M.J.: Amass: Archive of motion capture as surface shapes. In: Proceedings of the IEEE/CVF international conference on computer vision. pp. 5442–5451 (2019)
33. Mandery, C., Terlemez, Ö., Do, M., Vahrenkamp, N., Asfour, T.: The kit whole-body human motion database. In: 2015 International Conference on Advanced Robotics (ICAR). pp. 329–336. IEEE (2015)
34. Martinez, J., Hossain, R., Romero, J., Little, J.J.: A simple yet effective baseline for 3d human pose estimation. In: Proceedings of the IEEE international conference on computer vision. pp. 2640–2649 (2017)
35. Menolotto, M., Komaris, D.S., Tedesco, S., O'Flynn, B., Walsh, M.: Motion capture technology in industrial applications: A systematic review. Sensors **20**(19), 5687 (2020)
36. Moon, G., Chang, J.Y., Lee, K.M.: Camera distance-aware top-down approach for 3d multi-person pose estimation from a single rgb image. In: Proceedings of the IEEE/CVF international conference on computer vision. pp. 10133–10142 (2019)
37. Müller, M., Röder, T., Clausen, M.: Efficient content-based retrieval of motion capture data. In: ACM SIGGRAPH 2005 Papers, pp. 677–685 (2005)

38. Onuma, K., Faloutsos, C., Hodgins, J.K.: Fmdistance: A fast and effective distance function for motion capture data. In: Eurographics (Short Papers). pp. 83–86 (2008)
39. Pavlakos, G., Malik, J., Kanazawa, A.: Human mesh recovery from multiple shots. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 1485–1495 (2022)
40. Pavllo, D., Feichtenhofer, C., Grangier, D., Auli, M.: 3d human pose estimation in video with temporal convolutions and semi-supervised training. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. pp. 7753–7762 (2019)
41. Perez, E., Strub, F., De Vries, H., Dumoulin, V., Courville, A.: Film: Visual reasoning with a general conditioning layer. In: Proceedings of the AAAI conference on artificial intelligence. vol. 32 (2018)
42. Petrovich, M., Black, M.J., Varol, G.: Action-conditioned 3d human motion synthesis with transformer vae. In: ICCV (2021)
43. Plappert, M., Mandery, C., Asfour, T.: The kit motion-language dataset. Big data **4**(4), 236–252 (2016)
44. Raab, S., Leibovitch, I., Li, P., Aberman, K., Sorkine-Hornung, O., Cohen-Or, D.: Modi: Unconditional motion synthesis from diverse data. arXiv:2206.08010 (2022)
45. Ramesh, A., Dhariwal, P., Nichol, A., Chu, C., Chen, M.: Hierarchical text-conditional image generation with clip latents, 2022. URL https://arxiv.org/abs/2204.06125 **7** (2022)
46. Ruiz, N., Li, Y., Jampani, V., Pritch, Y., Rubinstein, M., Aberman, K.: Dreambooth: Fine tuning text-to-image diffusion models for subject-driven generation. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 22500–22510 (2023)
47. Saharia, C., Chan, W., Chang, H., Lee, C., Ho, J., Salimans, T., Fleet, D., Norouzi, M.: Palette: Image-to-image diffusion models. In: ACM SIGGRAPH 2022 Conference Proceedings. pp. 1–10 (2022)
48. Shan, W., Liu, Z., Zhang, X., Wang, S., Ma, S., Gao, W.: P-stmo: Pre-trained spatial temporal many-to-one model for 3d human pose estimation. In: European Conference on Computer Vision. pp. 461–478. Springer (2022)
49. Song, J., Meng, C., Ermon, S.: Denoising diffusion implicit models. arXiv preprint arXiv:2010.02502 (2020)
50. Sun, G., Wong, Y., Cheng, Z., Kankanhalli, M.S., Geng, W., Li, X.: Deepdance: music-to-dance motion choreography with adversarial learning. Transactions on Multimedia (2020)
51. Sun, X., Xiao, B., Wei, F., Liang, S., Wei, Y.: Integral human pose regression. In: Proceedings of the European conference on computer vision (ECCV). pp. 529–545 (2018)
52. Tevet, G., Gordon, B., Hertz, A., Bermano, A.H., Cohen-Or, D.: Motionclip: Exposing human motion generation to clip space. arXiv:2203.08063 (2022)
53. Tevet, G., Raab, S., Gordon, B., Shafir, Y., Cohen-Or, D., Bermano, A.H.: Human motion diffusion model. arXiv:2209.14916 (2022)
54. Tseng, J., Castellon, R., Liu, K.: Edge: Editable dance generation from music. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 448–458 (2023)
55. Wan, Z., Li, Z., Tian, M., Liu, J., Yi, S., Li, H.: Encoder-decoder with multi-level attention for 3d human shape and pose estimation. In: Proceedings of the IEEE/CVF International Conference on Computer Vision. pp. 13033–13042 (2021)

56. Wang, J., Yan, S., Dai, B., Lin, D.: Scene-aware generative network for human motion synthesis. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 12206–12215 (2021)
57. Wang, Z., Chen, Y., Liu, T., Zhu, Y., Liang, W., Huang, S.: Humanise: Language-conditioned human motion generation in 3d scenes. Advances in Neural Information Processing Systems **35**, 14959–14971 (2022)
58. Xie, X., Zhou, P., Li, H., Lin, Z., Yan, S.: Adan: Adaptive nesterov momentum algorithm for faster optimizing deep models. arXiv preprint arXiv:2208.06677 (2022)
59. Xu, X., Wang, Z., Zhang, G., Wang, K., Shi, H.: Versatile diffusion: Text, images and variations all in one diffusion model. In: Proceedings of the IEEE/CVF International Conference on Computer Vision. pp. 7754–7765 (2023)
60. Yan, S., Li, Z., Xiong, Y., Yan, H., Lin, D.: Convolutional sequence generation for skeleton-based action synthesis. In: ICCV (2019)
61. Ye, V., Pavlakos, G., Malik, J., Kanazawa, A.: Decoupling human and camera motion from videos in the wild. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 21222–21232 (2023)
62. Yi, H., Liang, H., Liu, Y., Cao, Q., Wen, Y., Bolkart, T., Tao, D., Black, M.J.: Generating holistic 3d human motion from speech. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 469–480 (2023)
63. Yuan, Y., Iqbal, U., Molchanov, P., Kitani, K., Kautz, J.: Glamr: Global occlusion-aware human mesh recovery with dynamic cameras. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. pp. 11038–11049 (2022)
64. Zhang, J., Yan, H., Xu, Z., Feng, J., Liew, J.H.: Magicavatar: Multimodal avatar generation and animation. arXiv preprint arXiv:2308.14748 (2023)
65. Zhang, J., Tu, Z., Yang, J., Chen, Y., Yuan, J.: Mixste: Seq2seq mixed spatio-temporal encoder for 3d human pose estimation in video. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. pp. 13232–13242 (2022)
66. Zhang, L., Rao, A., Agrawala, M.: Adding conditional control to text-to-image diffusion models (2023)
67. Zhang, M., Cai, Z., Pan, L., Hong, F., Guo, X., Yang, L., Liu, Z.: Motiondiffuse: Text-driven human motion generation with diffusion model. arXiv:2208.15001 (2022)
68. Zheng, C., Zhu, S., Mendieta, M., Yang, T., Chen, C., Ding, Z.: 3d human pose estimation with spatial and temporal transformers. In: Proceedings of the IEEE/CVF International Conference on Computer Vision. pp. 11656–11665 (2021)
69. Zhi, Y., Cun, X., Chen, X., Shen, X., Guo, W., Huang, S., Gao, S.: Livelyspeaker: Towards semantic-aware co-speech gesture generation. In: Proceedings of the IEEE/CVF International Conference on Computer Vision. pp. 20807–20817 (2023)
70. Zhou, K., Han, X., Jiang, N., Jia, K., Lu, J.: Hemlets pose: Learning part-centric heatmap triplets for accurate 3d human pose estimation. In: Proceedings of the IEEE/CVF international conference on computer vision. pp. 2344–2353 (2019)
71. Zhu, W., Ma, X., Liu, Z., Liu, L., Wu, W., Wang, Y.: Motionbert: A unified perspective on learning human motion representations. In: Proceedings of the IEEE/CVF International Conference on Computer Vision. pp. 15085–15099 (2023)